

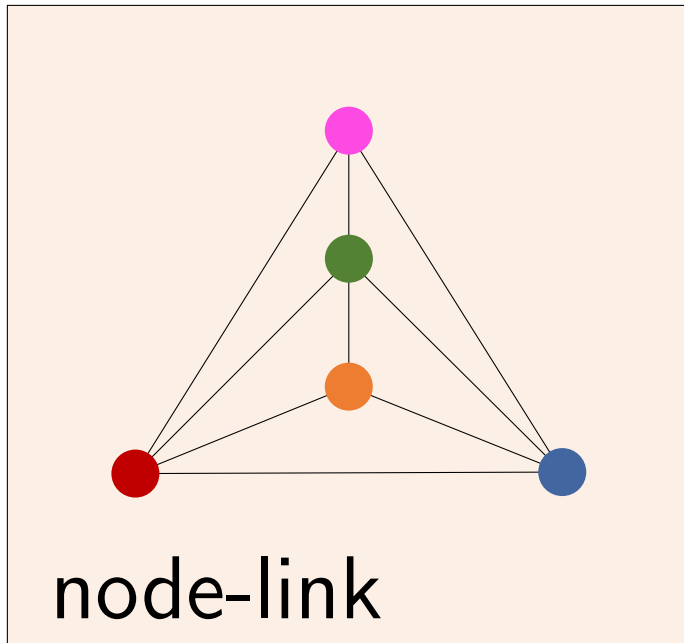
An algorithm for accurate and simple-looking metaphorical maps

Eleni Katsanou, Tamara Mchedlidze,
Antonios Symvonis, Thanos Tolias

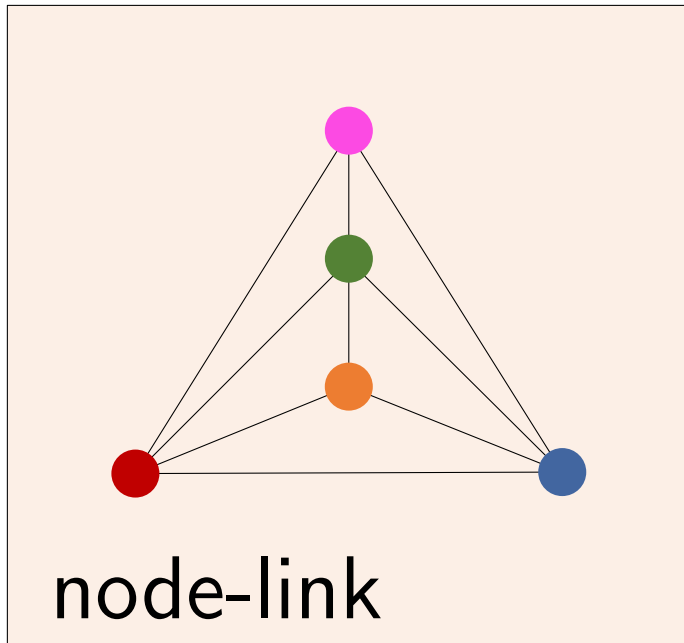
33rd International Symposium on Graph Drawing and Network
Visualization (GD 2025)

Popular Visualization Paradigms

Popular Visualization Paradigms



Popular Visualization Paradigms

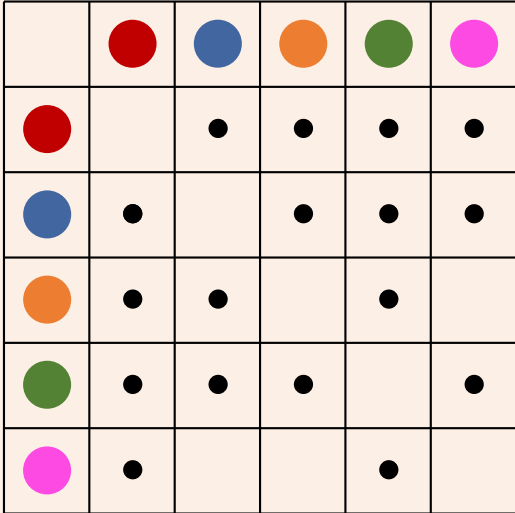
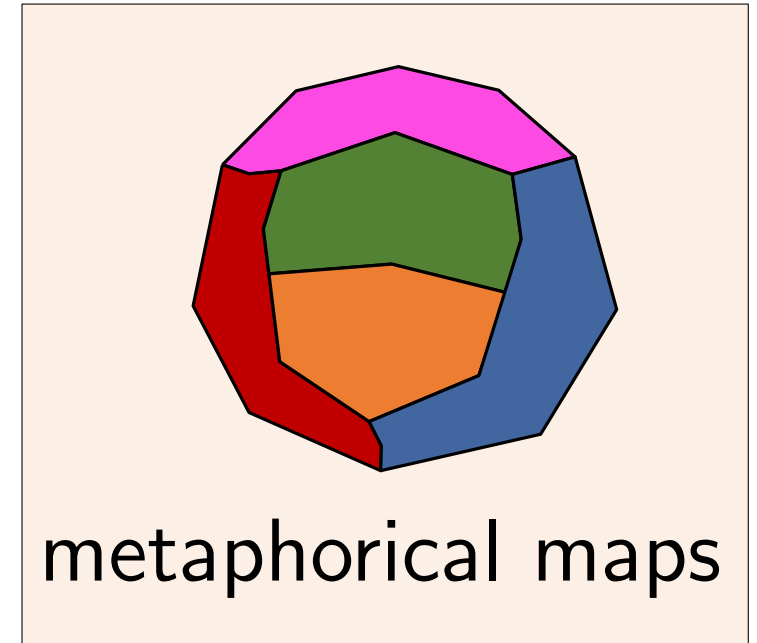
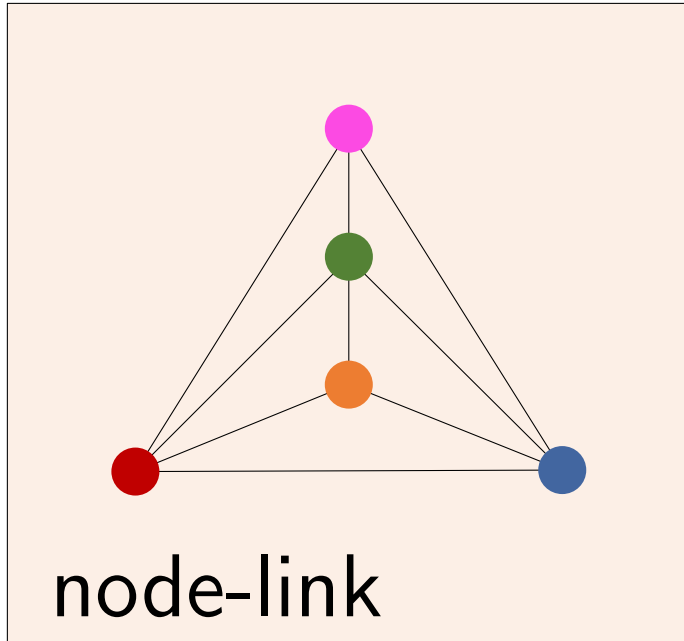


A matrix visualization of the graph's adjacency matrix. The matrix is a 6x6 grid where the top row and left column contain the node labels (red, blue, orange, green, pink) represented by colored circles. The cells in the matrix contain black dots representing the edges between the nodes. The matrix is symmetric, reflecting the undirected nature of the graph.

	red	blue	orange	green	pink
red		•	•	•	•
blue	•		•	•	•
orange	•	•		•	•
green	•	•	•		•
pink	•	•	•	•	

matrix

Popular Visualization Paradigms



matrix

	Red	Blue	Orange	Green	Pink
Red		•	•	•	•
Blue	•		•	•	•
Orange	•	•		•	
Green	•	•	•		•
Pink	•			•	

A matrix diagram showing a 6x6 grid. The first row and column are headers with colored nodes: Red, Blue, Orange, Green, Pink. The cells contain black dots representing connections between the nodes. The connections are: Red-Blue, Red-Orange, Red-Green, Red-Pink, Blue-Orange, Blue-Green, Blue-Pink, Orange-Green, Orange-Pink, Green-Pink.

Advantages of metaphorical maps

Advantages of metaphorical maps

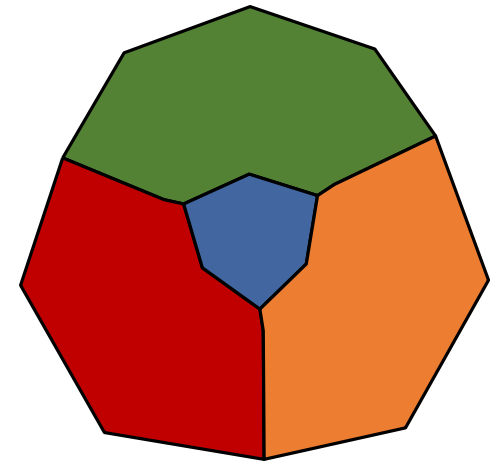
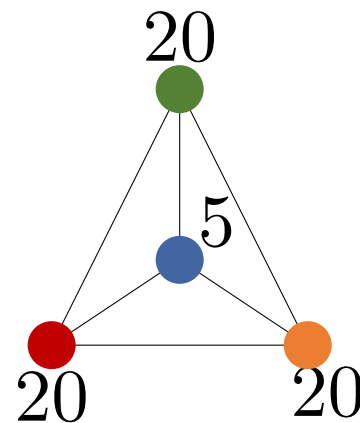
- ▶ more enjoyable than node-link diagrams (Saket, Scheidegger, and Kobourov. 2016)

Advantages of metaphorical maps

- ▶ more enjoyable than node-link diagrams (Saket, Scheidegger, and Kobourov. 2016)
- ▶ we are familiar with geographic maps

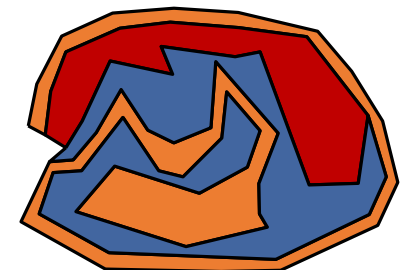
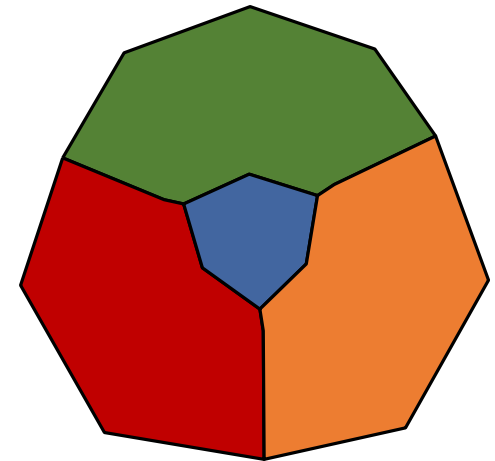
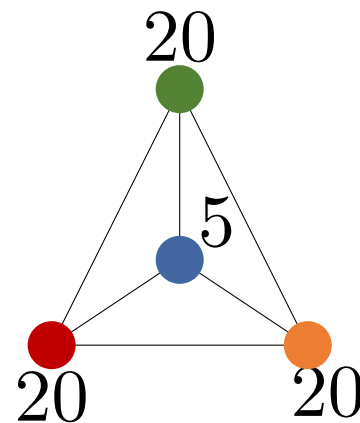
Advantages of metaphorical maps

- ▶ more enjoyable than node-link diagrams (Saket, Scheidegger, and Kobourov. 2016)
- ▶ we are familiar with geographic maps
- ▶ display the weights of the graph's vertices by adjusting the area of each polygon



Advantages of metaphorical maps

- ▶ more enjoyable than node-link diagrams (Saket, Scheidegger, and Kobourov. 2016)
- ▶ we are familiar with geographic maps
- ▶ display the weights of the graph's vertices by adjusting the area of each polygon
- ▶ **Criterion:** simple regions' outlines
 - We want to be able to compare areas



Map-like representations with simple regions

- ▶ Mchedlidze and Schnorr. 2022
 - force-based algorithm

Map-like representations with simple regions

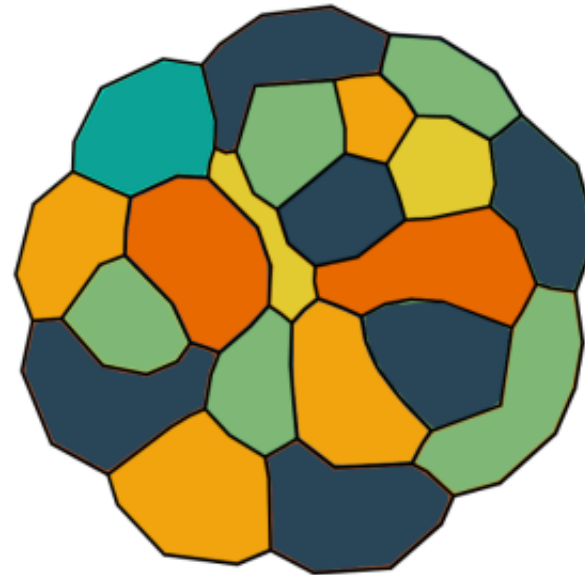
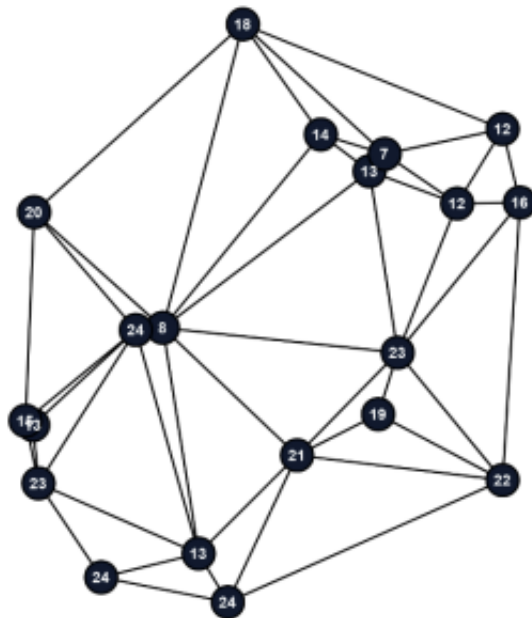
- ▶ Mchedlidze and Schnorr. 2022
 - force-based algorithm
 - visually simple

Map-like representations with simple regions

- ▶ Mchedlidze and Schnorr. 2022
 - force-based algorithm
 - visually simple
 - cartographic error up to 30 %

Map-like representations with simple regions

- ▶ Mchedlidze and Schnorr. 2022
 - force-based algorithm
 - visually simple
 - cartographic error up to 30 %



- ▶ extension of the Mchedlidze and Schnorr force-directed algorithm to achieve **simple** metaphorical maps with **near zero cartographical error**

- ▶ extension of the Mchedlidze and Schnorr force-directed algorithm to achieve **simple** metaphorical maps with **near zero cartographical error**
 - region stiffness

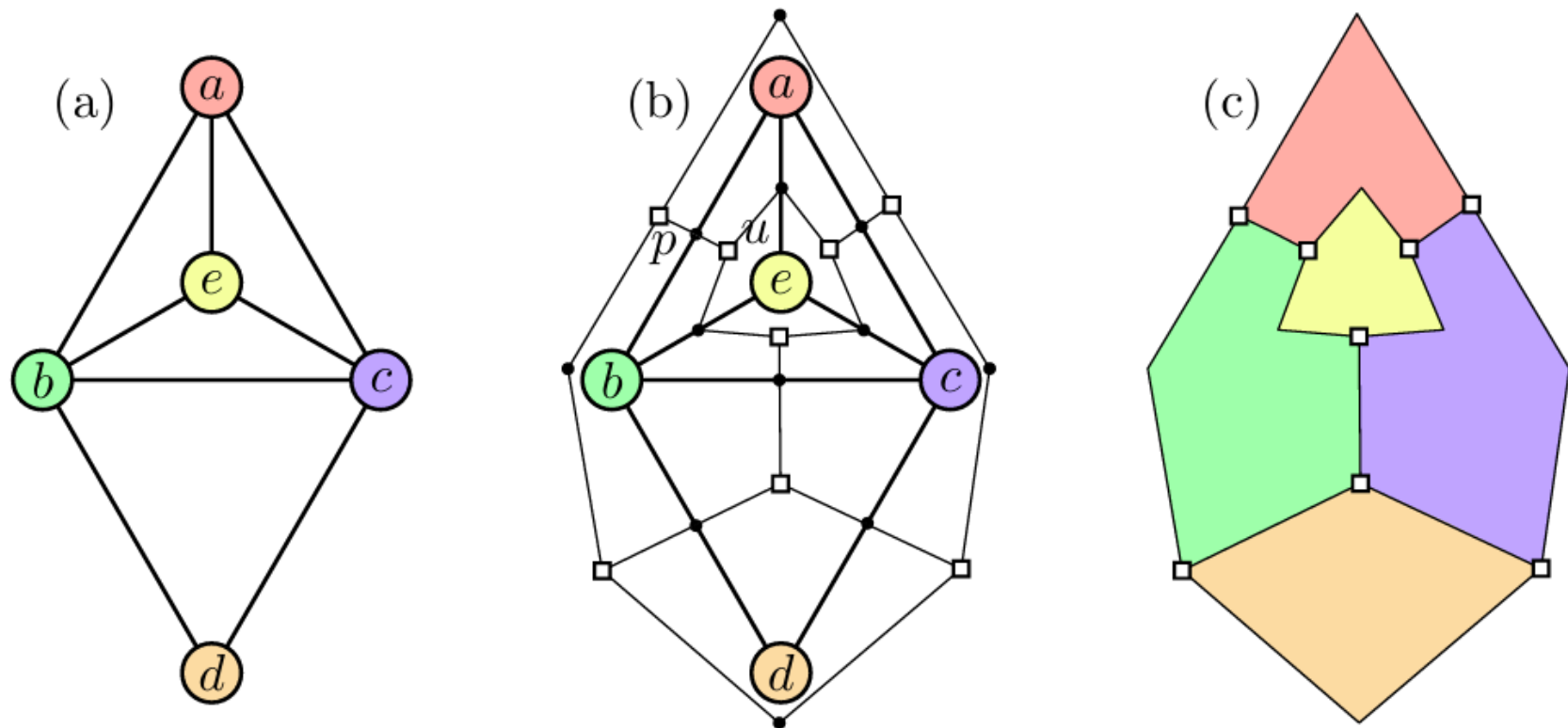
- ▶ extension of the Mchedlidze and Schnorr force-directed algorithm to achieve **simple** metaphorical maps with **near zero cartographical error**
 - region stiffness
 - different pressure coefficients around the polygon

- ▶ extension of the Mchedlidze and Schnorr force-directed algorithm to achieve **simple** metaphorical maps with **near zero cartographical error**
 - region stiffness
 - different pressure coefficients around the polygon
 - non-triangulated graphs

- ▶ extension of the Mchedlidze and Schnorr force-directed algorithm to achieve **simple** metaphorical maps with **near zero cartographical error**
 - region stiffness
 - different pressure coefficients around the polygon
 - non-triangulated graphs
- ▶ experimental evaluation

The Mchedlidze & Schnorr Algorithm

- Step 1: Transform graph into metaphorical map

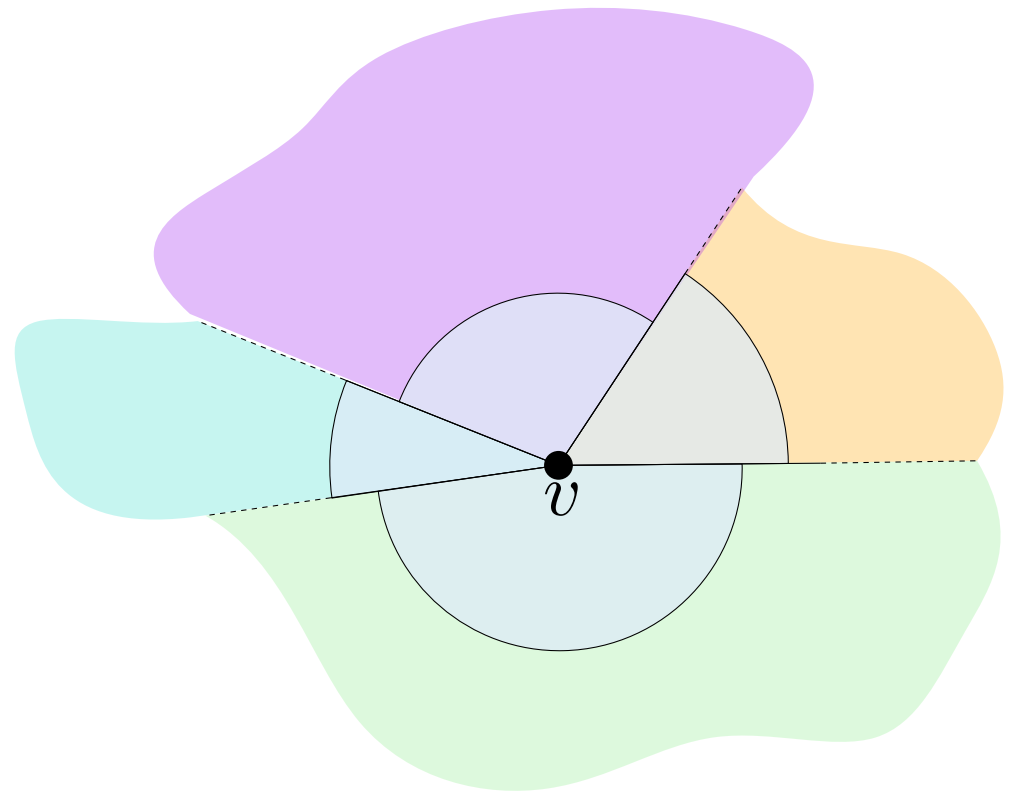


The Mchedlidze & Schnorr Algorithm

- ▶ Step 2: Apply forces iteratively

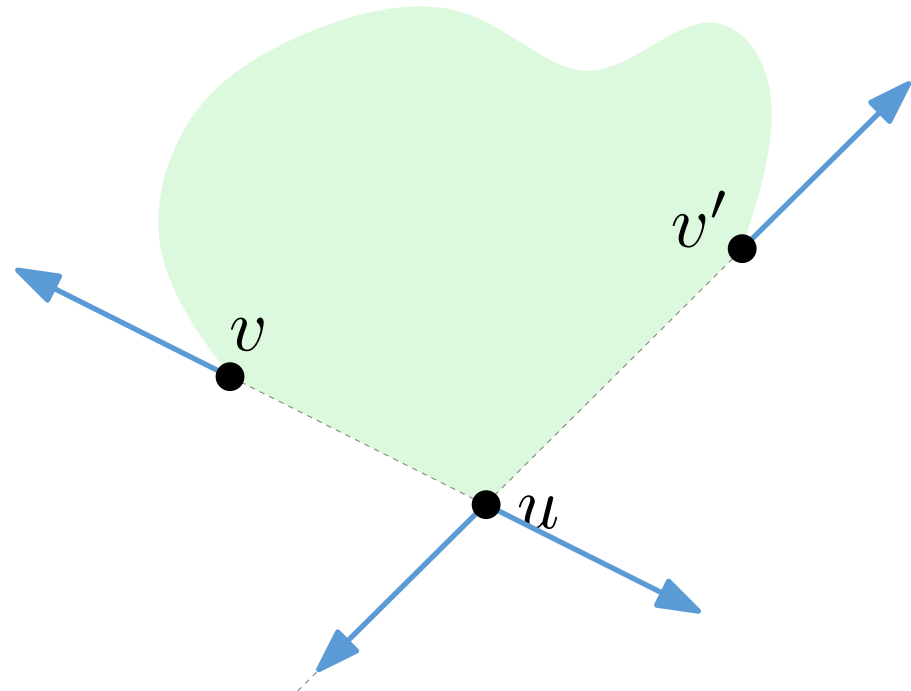
The Mchedlidze & Schnorr Algorithm

- Step 2: Apply forces iteratively
 - Angular resolution



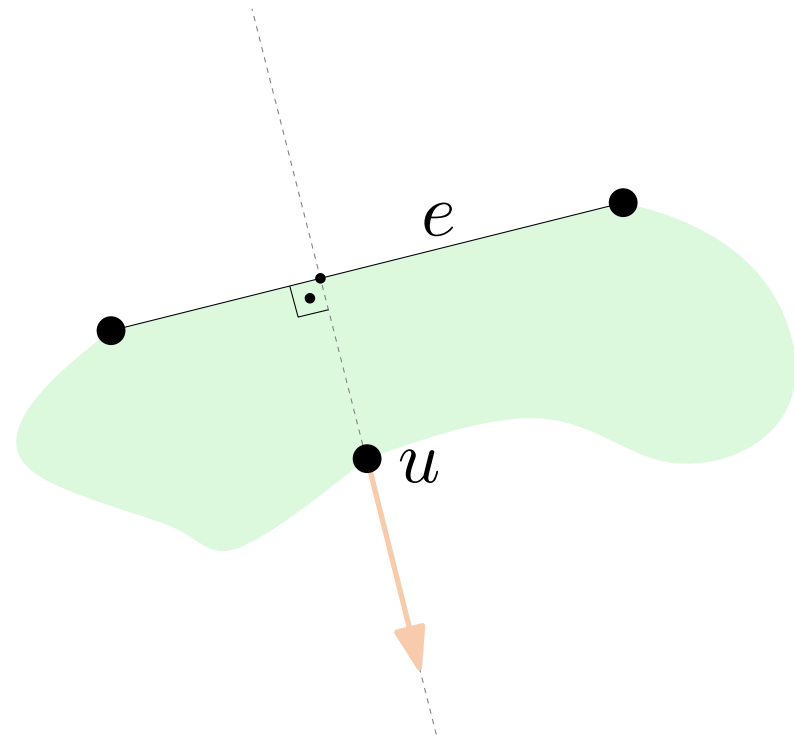
The Mchedlidze & Schnorr Algorithm

- ▶ Step 2: Apply forces iteratively
 - Angular resolution
 - Vertex-vertex repulsion



The Mchedlidze & Schnorr Algorithm

- ▶ Step 2: Apply forces iteratively
 - Angular resolution
 - Vertex-vertex repulsion
 - Vertex-edge repulsion



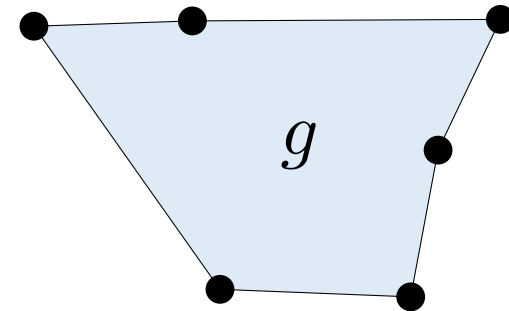
The Mchedlidze & Schnorr Algorithm

- Step 2: Apply forces iteratively
 - Angular resolution
 - Vertex-vertex repulsion
 - Vertex-edge repulsion
 - Pressure force

$$P(g) = \frac{w(g)}{A(g)} \frac{\sum_{f \in F} A(f)}{\sum_{f \in F} w(f)}$$

$$P(g_{outer}) = 1$$

$$\mathbf{F}(e, g) = 3P(g) \frac{\ell(e)}{\text{circ}(g)} \hat{\mathbf{r}}$$



$w(g)$: weight of the corresponding vertex

$A(g)$: area of region g

The Mchedlidze & Schnorr Algorithm

- Step 2: Apply forces iteratively
 - Angular resolution
 - Vertex-vertex repulsion
 - Vertex-edge repulsion
 - Pressure force

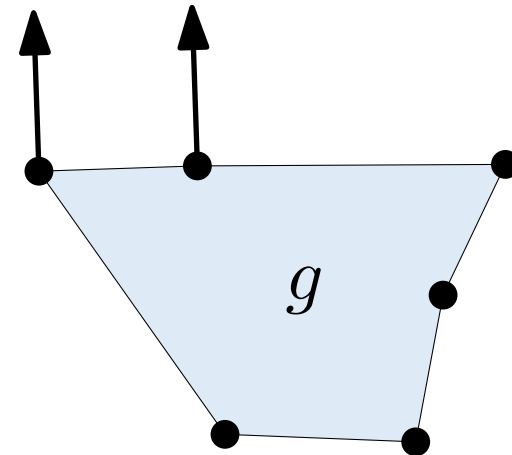
$$P(g) = \frac{w(g)}{A(g)} \frac{\sum_{f \in F} A(f)}{\sum_{f \in F} w(f)}$$

$$P(g_{outer}) = 1$$

$$\mathbf{F}(e, g) = 3P(g) \frac{\ell(e)}{\text{circ}(g)} \hat{\mathbf{r}}$$

$w(g)$: weight of the corresponding vertex

$A(g)$: area of region g



The Mchedlidze & Schnorr Algorithm

- Step 2: Apply forces iteratively
 - Angular resolution
 - Vertex-vertex repulsion
 - Vertex-edge repulsion
 - Pressure force

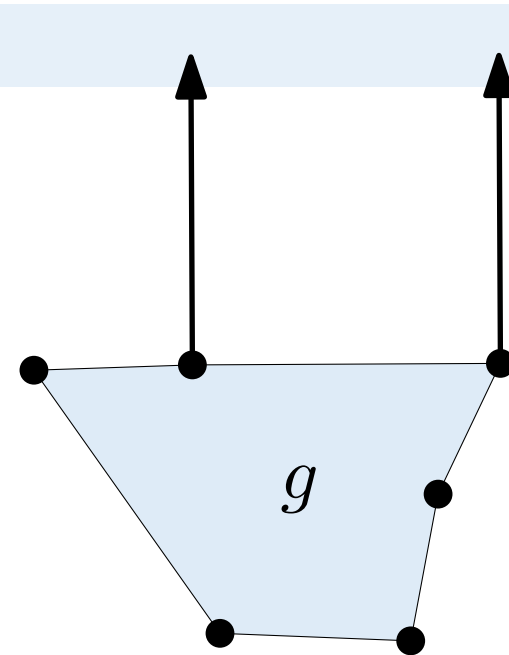
$$P(g) = \frac{w(g)}{A(g)} \frac{\sum_{f \in F} A(f)}{\sum_{f \in F} w(f)}$$

$$P(g_{outer}) = 1$$

$$\mathbf{F}(e, g) = 3P(g) \frac{\ell(e)}{\text{circ}(g)} \hat{\mathbf{r}}$$

$w(g)$: weight of the corresponding vertex

$A(g)$: area of region g



The Mchedlidze & Schnorr Algorithm

- ▶ Step 2: Apply forces iteratively
 - Angular resolution
 - Vertex-vertex repulsion
 - Vertex-edge repulsion
 - Pressure force

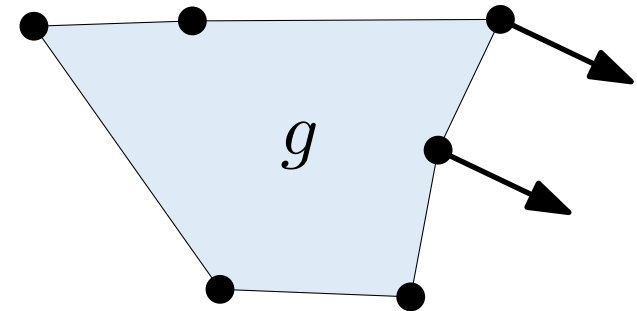
$$P(g) = \frac{w(g)}{A(g)} \frac{\sum_{f \in F} A(f)}{\sum_{f \in F} w(f)}$$

$$P(g_{outer}) = 1$$

$$\mathbf{F}(e, g) = 3P(g) \frac{\ell(e)}{\text{circ}(g)} \hat{\mathbf{r}}$$

$w(g)$: weight of the corresponding vertex

$A(g)$: area of region g



The Mchedlidze & Schnorr Algorithm

- Step 2: Apply forces iteratively
 - Angular resolution
 - Vertex-vertex repulsion
 - Vertex-edge repulsion
 - Pressure force

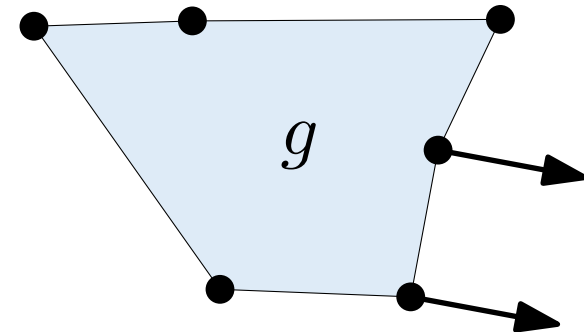
$$P(g) = \frac{w(g)}{A(g)} \frac{\sum_{f \in F} A(f)}{\sum_{f \in F} w(f)}$$

$$P(g_{outer}) = 1$$

$$\mathbf{F}(e, g) = 3P(g) \frac{\ell(e)}{\text{circ}(g)} \hat{\mathbf{r}}$$

$w(g)$: weight of the corresponding vertex

$A(g)$: area of region g



The Mchedlidze & Schnorr Algorithm

- Step 2: Apply forces iteratively
 - Angular resolution
 - Vertex-vertex repulsion
 - Vertex-edge repulsion
 - Pressure force

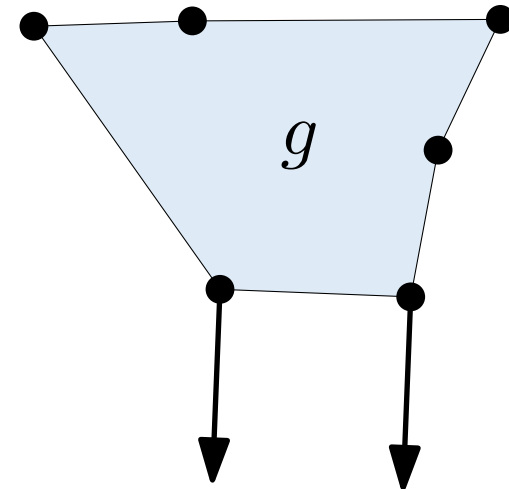
$$P(g) = \frac{w(g)}{A(g)} \frac{\sum_{f \in F} A(f)}{\sum_{f \in F} w(f)}$$

$$P(g_{outer}) = 1$$

$$\mathbf{F}(e, g) = 3P(g) \frac{\ell(e)}{\text{circ}(g)} \hat{\mathbf{r}}$$

$w(g)$: weight of the corresponding vertex

$A(g)$: area of region g



The Mchedlidze & Schnorr Algorithm

- ▶ Step 2: Apply forces iteratively
 - Angular resolution
 - Vertex-vertex repulsion
 - Vertex-edge repulsion
 - Pressure force

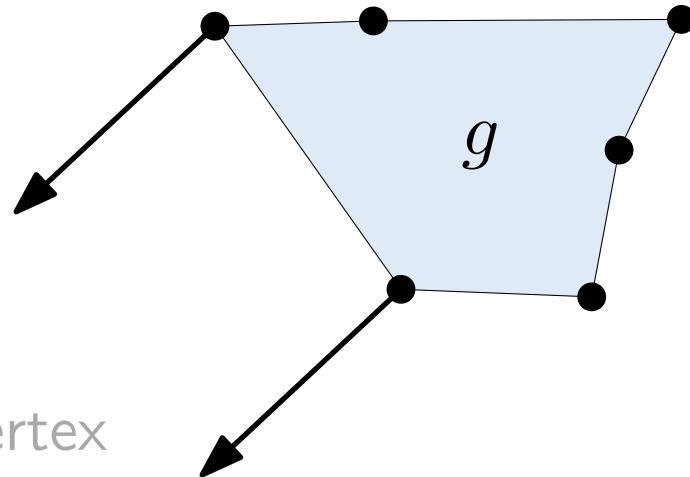
$$P(g) = \frac{w(g)}{A(g)} \frac{\sum_{f \in F} A(f)}{\sum_{f \in F} w(f)}$$

$$P(g_{outer}) = 1$$

$$\mathbf{F}(e, g) = 3P(g) \frac{\ell(e)}{\text{circ}(g)} \hat{\mathbf{r}}$$

$w(g)$: weight of the corresponding vertex

$A(g)$: area of region g



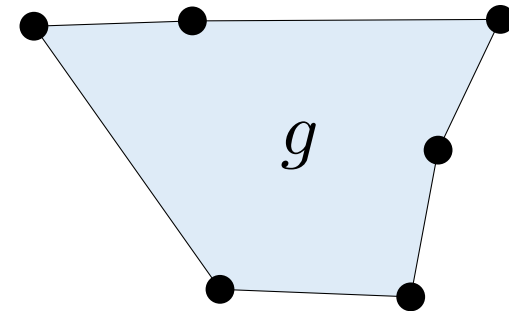
The Mchedlidze & Schnorr Algorithm

- Step 2: Apply forces iteratively
 - Angular resolution
 - Vertex-vertex repulsion
 - Vertex-edge repulsion
 - Pressure force

$$P(g) = \frac{w(g)}{A(g)} \frac{\sum_{f \in F} A(f)}{\sum_{f \in F} w(f)}$$

$$P(g_{outer}) = 1$$

$$\mathbf{F}(e, g) = 3P(g) \frac{\ell(e)}{\text{circ}(g)} \hat{\mathbf{r}}$$



$w(g)$: weight of the corresponding vertex

$A(g)$: area of region g

Our Metaphorical Map Generation Algorithm

6 - 1

Our Metaphorical Map Generation Algorithm

6 - 2

The Stiffness of each Map Region

The Stiffness of each Map Region

Some regions don't respond well to the pressure forces. They either inflate too much, or deflate too little.

The Stiffness of each Map Region

Some regions don't respond well to the pressure forces. They either inflate too much, or deflate too little.

- ▶ **Idea:** polygons can act as balloons with different elasticity

The Stiffness of each Map Region

Some regions don't respond well to the pressure forces. They either inflate too much, or deflate too little.

- ▶ **Idea:** polygons can act as balloons with different elasticity
- ▶ New pressure force at iteration i :

The Stiffness of each Map Region

Some regions don't respond well to the pressure forces. They either inflate too much, or deflate too little.

- ▶ **Idea:** polygons can act as balloons with different elasticity
- ▶ New pressure force at iteration i :

$$\mathbf{F}'_i(e, g) = s_i(g) \cdot \mathbf{F}_i(e, g)$$

$$s_i(g) = s_{i-1}(g) + \alpha \cdot 0.02$$

$$\alpha = \begin{cases} -1, & \text{if region is **over-pressured**} \\ 0, & \text{if region has correct pressure} \\ 1, & \text{if region is **under-pressured**} \end{cases}$$

Our Metaphorical Map Generation Algorithm

6 - 7

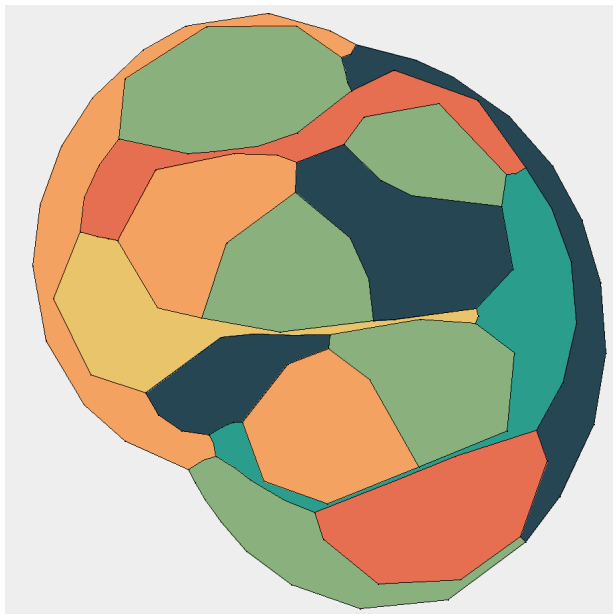
Improving the Visual Complexity

Improving the Visual Complexity

Applying stiffness created long, narrow passages.

Improving the Visual Complexity

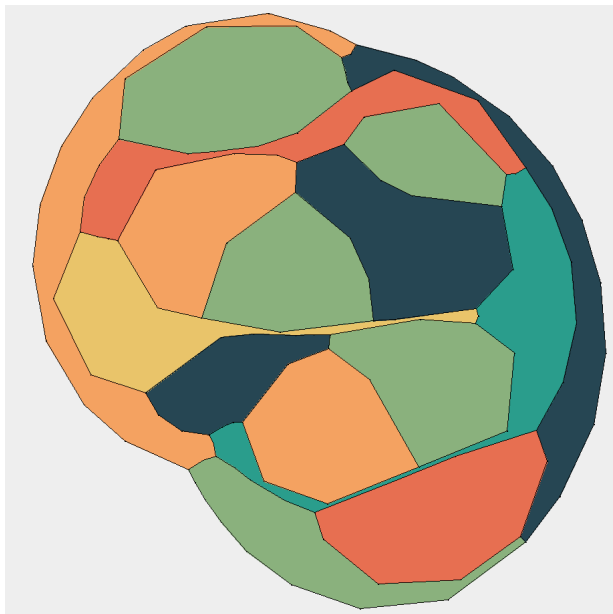
Applying stiffness created long, narrow passages.



Improving the Visual Complexity

Applying stiffness created long, narrow passages.

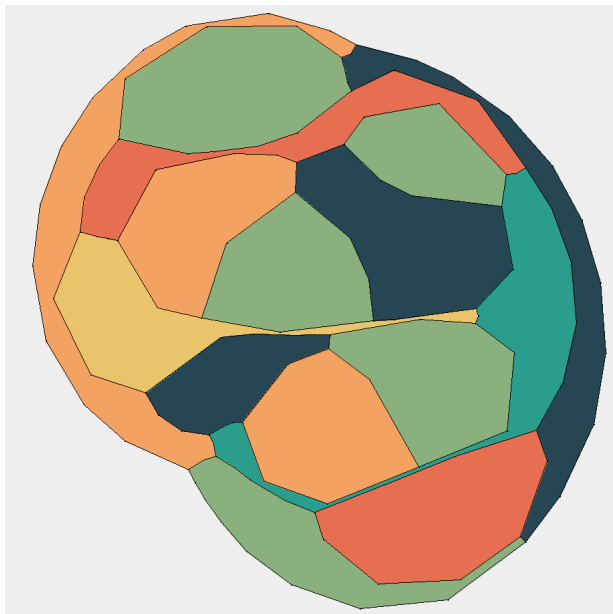
- **Idea:** redistribute pressure inside each region



Improving the Visual Complexity

Applying stiffness created long, narrow passages.

- ▶ **Idea:** redistribute pressure inside each region
 - **Narrow passages** → vertices there get **more pressure**, pushing passages open.
 - **Wide areas** → vertices there get **less pressure**.



Our Metaphorical Map Generation Algorithm

6 - 12

How narrow passages are detected

How narrow passages are detected

For each vertex u we want to identify the edge that is the closest out of those opposite to it.

“**opposite edge**”: euclidean distance is **much** smaller than boundary distance

How narrow passages are detected

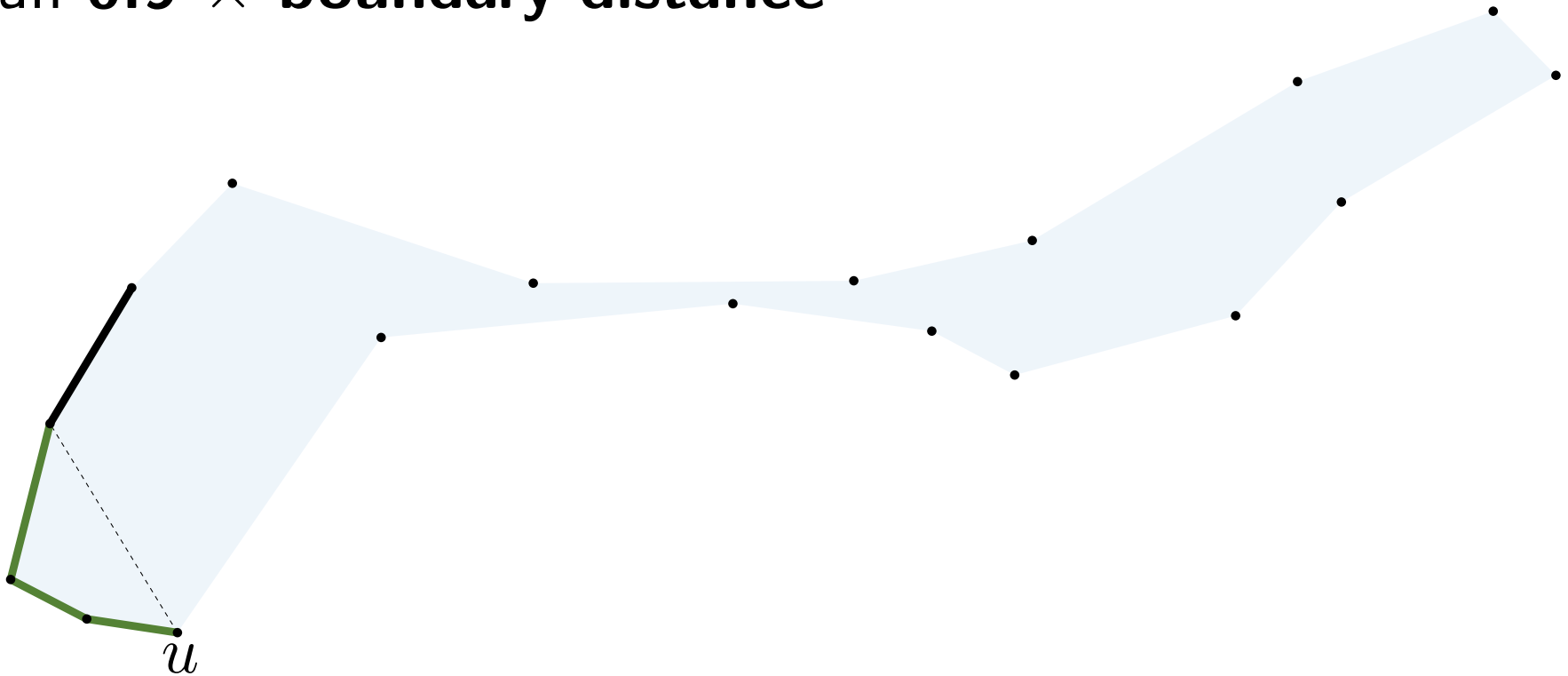
For each vertex u we want to identify the edge that is the closest out of those opposite to it.

“**opposite edge**”: euclidean distance is smaller than **0.9** $\times$ **boundary distance**

How narrow passages are detected

For each vertex u we want to identify the edge that is the closest out of those opposite to it.

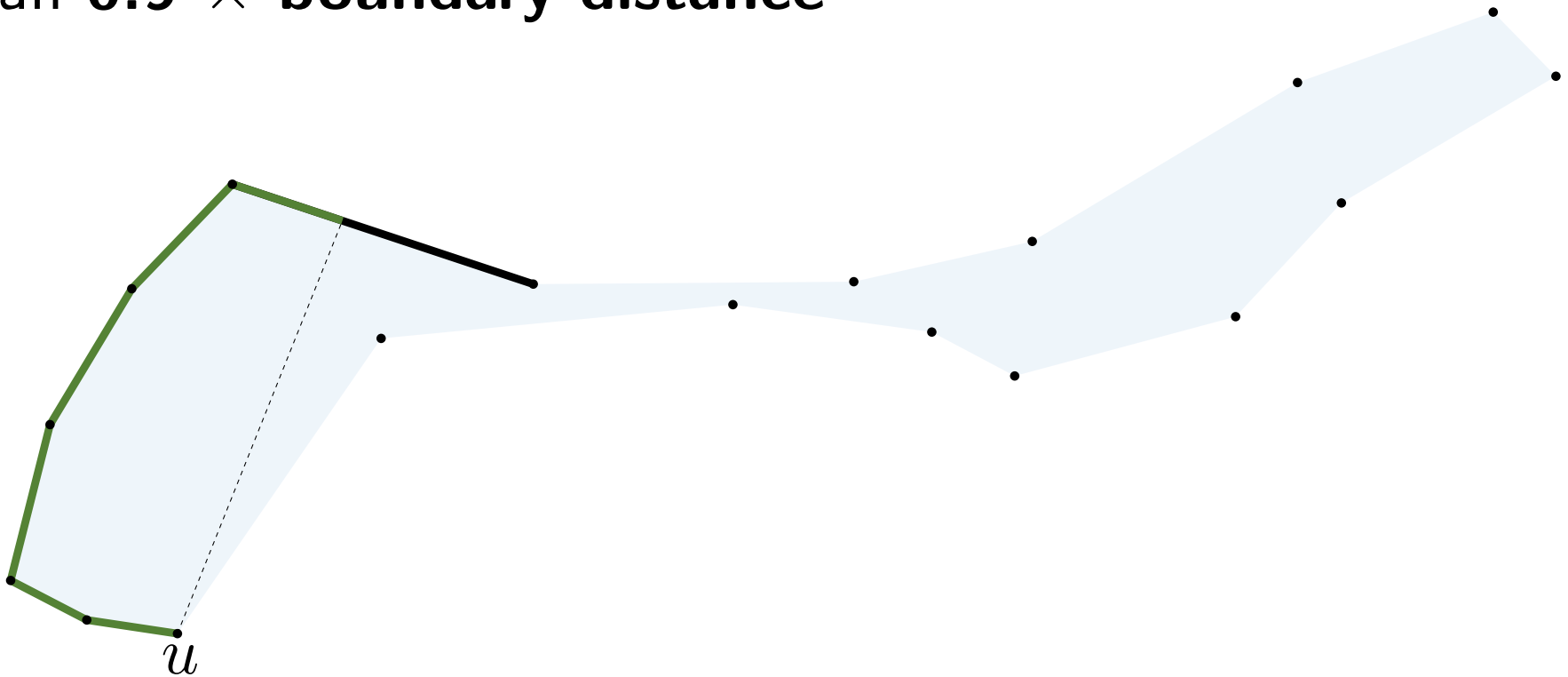
“**opposite edge**”: euclidean distance is smaller than $0.9 \times$ **boundary distance**



How narrow passages are detected

For each vertex u we want to identify the edge that is the closest out of those opposite to it.

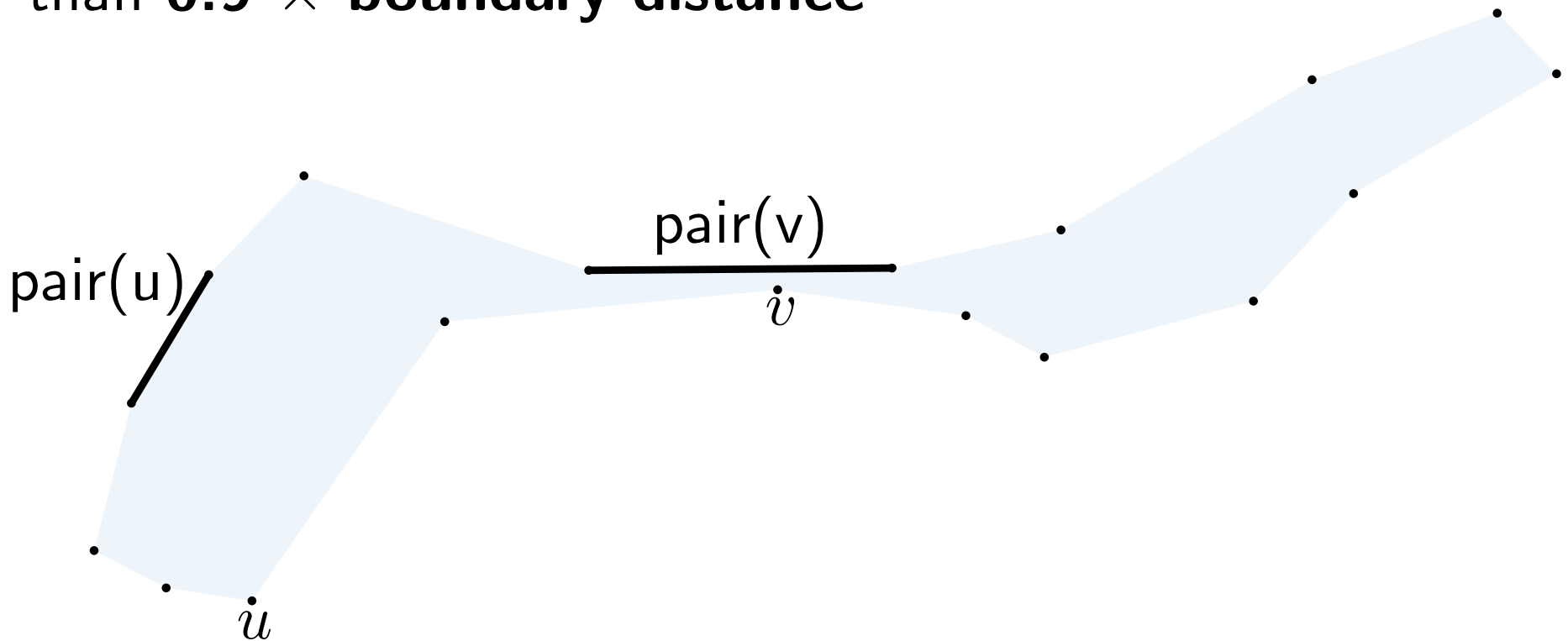
“**opposite edge**”: euclidean distance is smaller than $0.9 \times$ **boundary distance**



How narrow passages are detected

For each vertex u we want to identify the edge that is the closest out of those opposite to it.

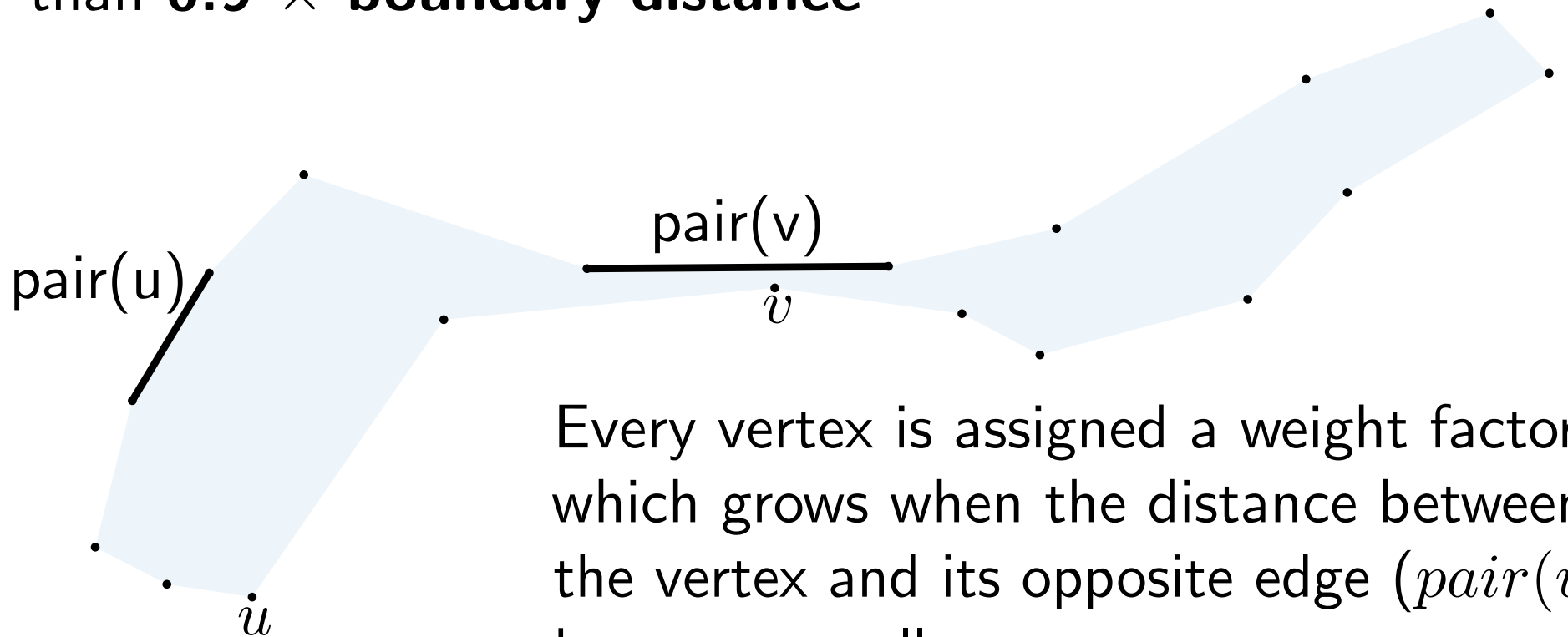
“**opposite edge**”: euclidean distance is smaller than $0.9 \times$ **boundary distance**



How narrow passages are detected

For each vertex u we want to identify the edge that is the closest out of those opposite to it.

“**opposite edge**”: euclidean distance is smaller than $0.9 \times$ **boundary distance**

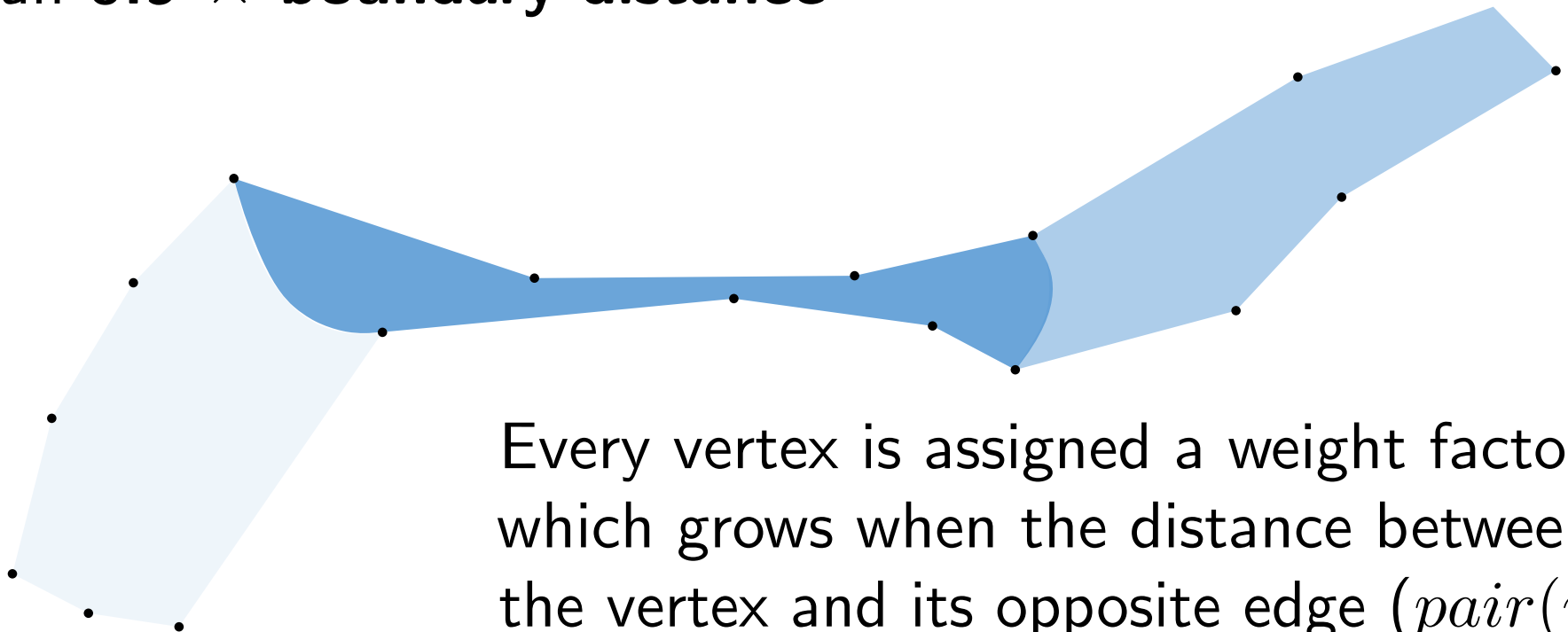


Every vertex is assigned a weight factor, which grows when the distance between the vertex and its opposite edge ($pair(u)$) becomes small.

How narrow passages are detected

For each vertex u we want to identify the edge that is the closest out of those opposite to it.

“**opposite edge**”: euclidean distance is smaller than $0.9 \times$ **boundary distance**

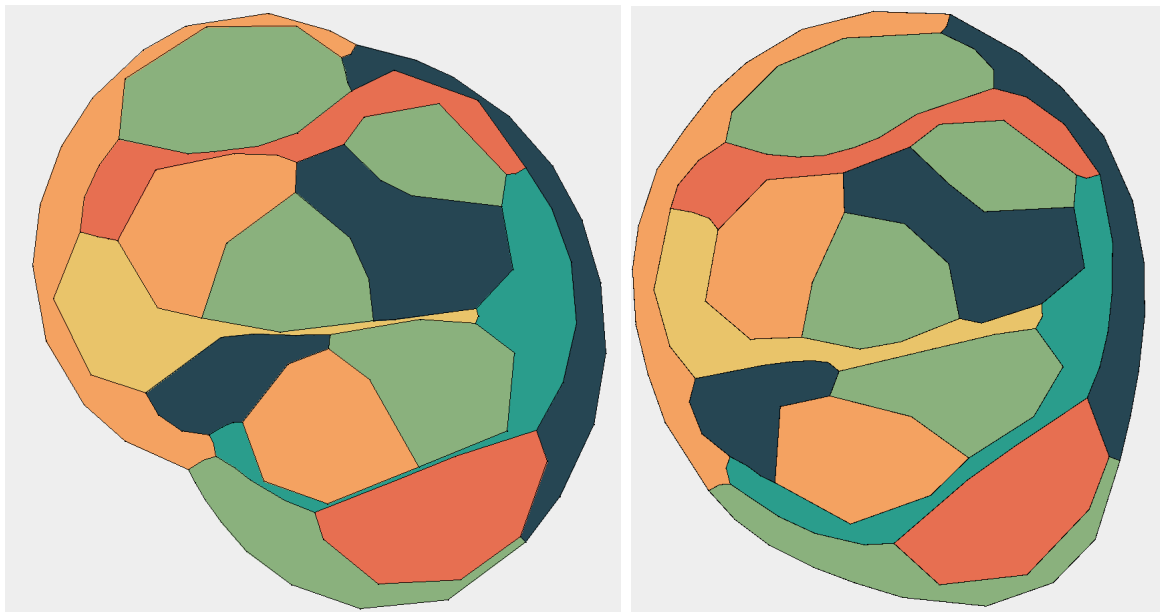


Every vertex is assigned a weight factor, which grows when the distance between the vertex and its opposite edge ($pair(u)$) becomes small.

Improving the Visual Complexity

Applying stiffness created long, narrow passages.

- **Idea:** redistribute pressure inside each region



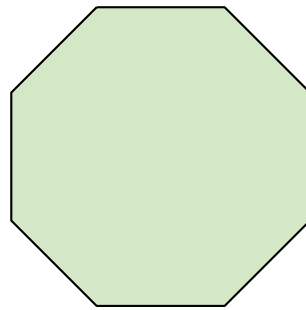
Quality Metrics

7 - 1

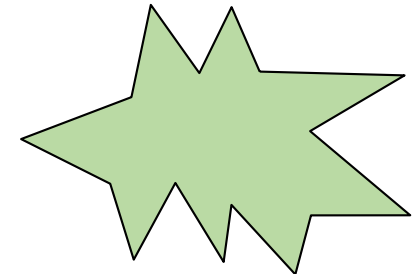
- ▶ Normalized cartographic error

- ▶ Normalized cartographic error
- ▶ Polygon Complexity (Brinkhoff, Kriegel, Schneider, and Braun. 1995)

- ▶ Normalized cartographic error
- ▶ Polygon Complexity (Brinkhoff, Kriegel, Schneider, and Braun. 1995)
 - frequency

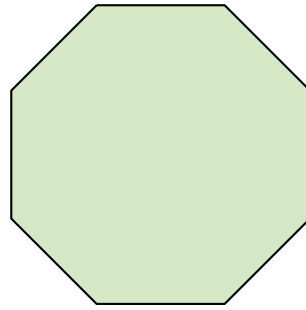


0 when convex
polygon

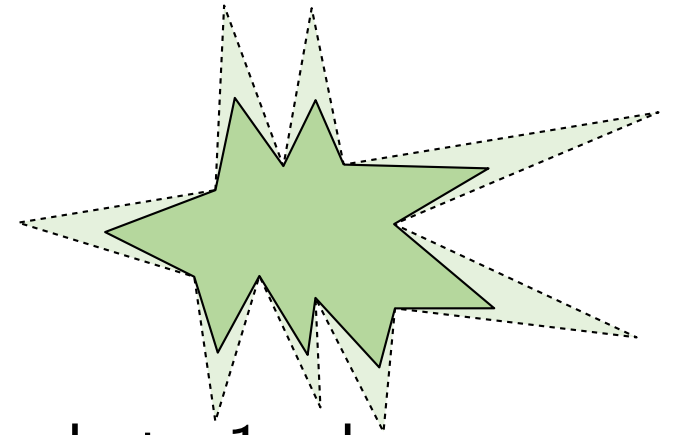


1 when half of the
internal angles are
concave

- ▶ Normalized cartographic error
- ▶ Polygon Complexity (Brinkhoff, Kriegel, Schneider, and Braun. 1995)
 - frequency
 - amplitude

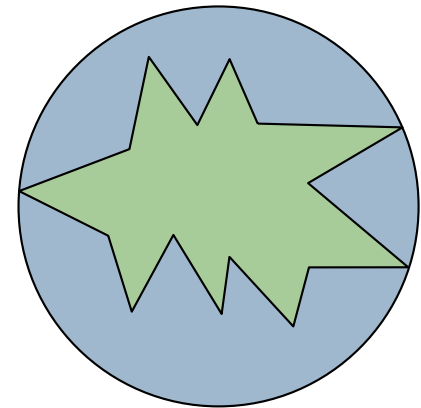


0 when convex
polygon



tends to 1 when
boundary gets longer

- ▶ Normalized cartographic error
- ▶ Polygon Complexity (Brinkhoff, Kriegel, Schneider, and Braun. 1995)
 - frequency
 - amplitude
 - convexity



- ▶ Normalized cartographic error
- ▶ Polygon Complexity (Brinkhoff, Kriegel, Schneider, and Braun. 1995)
 - frequency
 - amplitude
 - convexity

$$\text{compl}(P) = 0.8 \cdot \text{ampl}(P) \cdot \text{freq}(P) + 0.2 \cdot \text{conv}(P)$$

Generation of test data (same as Mchedlidze & Schnorr)

Generation of test data (same as Mchedlidze & Schnorr)

Input:

Generation of test data (same as Mchedlidze & Schnorr)

Input:

- ▶ number of vertices n

Generation of test data (same as Mchedlidze & Schnorr)

Input:

- ▶ number of vertices n
- ▶ weight ratio w : ratio of maximum to minimum vertex weight

Generation of test data (same as Mchedlidze & Schnorr)

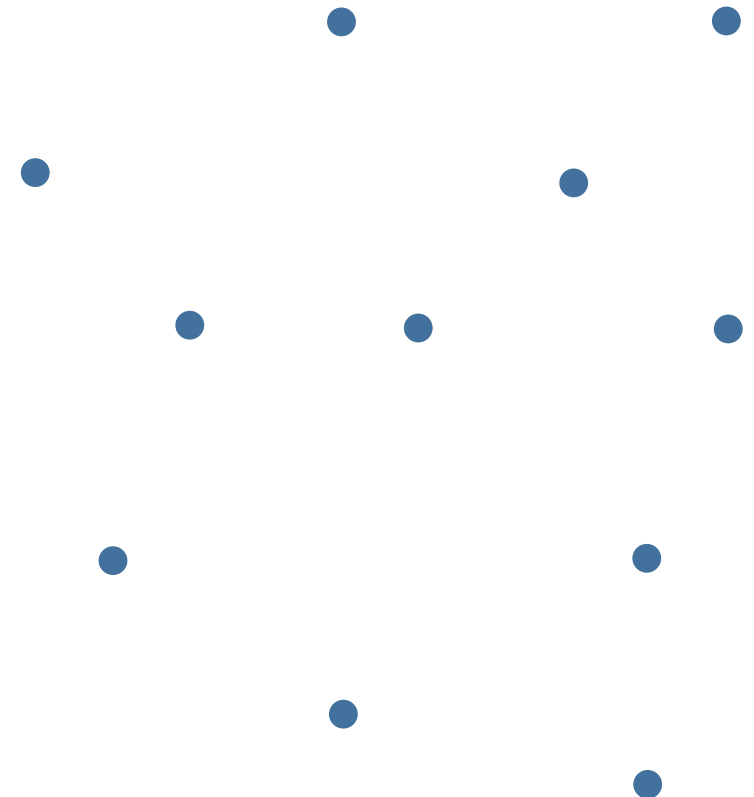
Input:

- ▶ number of vertices n
- ▶ weight ratio w : ratio of maximum to minimum vertex weight
- ▶ nesting ratio $nest$: fraction of the points to nest inside triangles

Generation of test data (same as Mchedlidze & Schnorr)

Input:

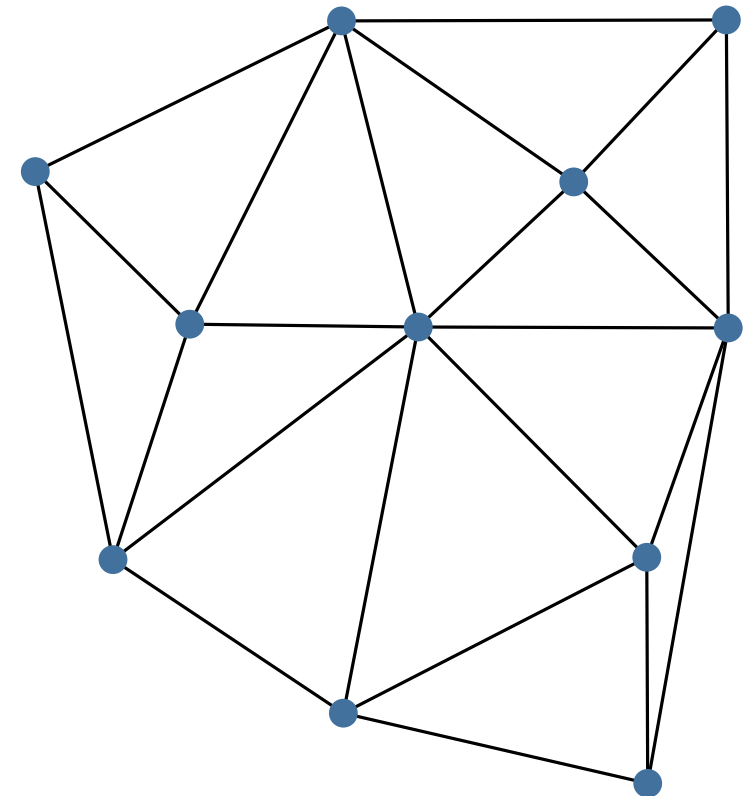
- ▶ number of vertices n
- ▶ weight ratio w : ratio of maximum to minimum vertex weight
- ▶ nesting ratio $nest$: fraction of the points to nest inside triangles



Generation of test data (same as Mchedlidze & Schnorr)

Input:

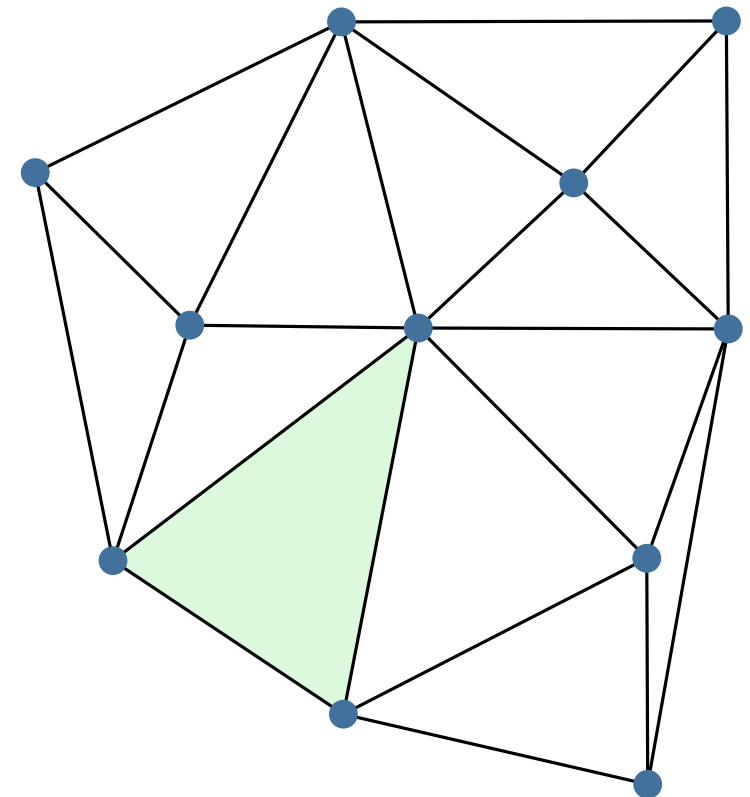
- ▶ number of vertices n
- ▶ weight ratio w : ratio of maximum to minimum vertex weight
- ▶ nesting ratio $nest$: fraction of the points to nest inside triangles



Generation of test data (same as Mchedlidze & Schnorr)

Input:

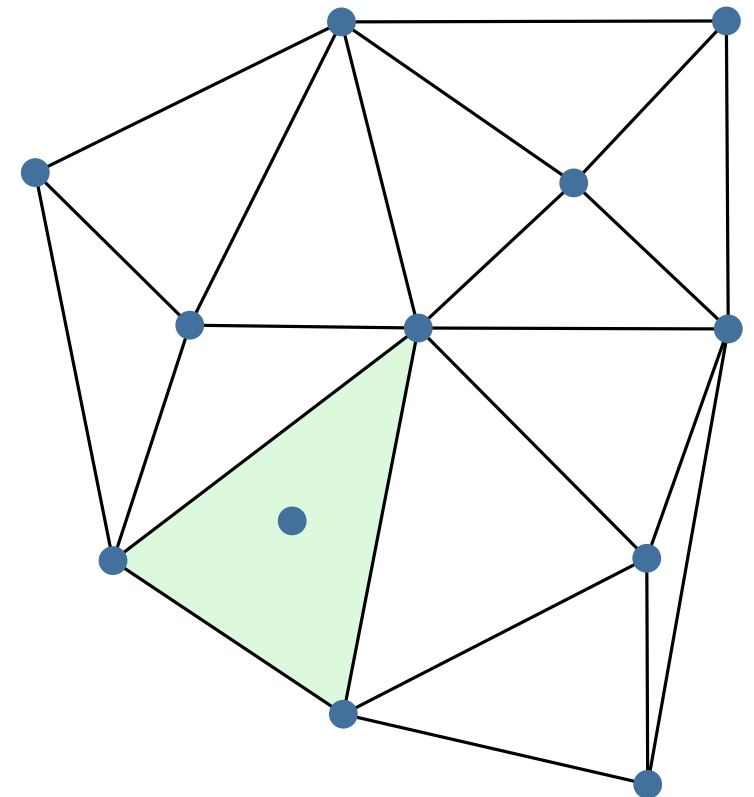
- ▶ number of vertices n
- ▶ weight ratio w : ratio of maximum to minimum vertex weight
- ▶ nesting ratio $nest$: fraction of the points to nest inside triangles



Generation of test data (same as Mchedlidze & Schnorr)

Input:

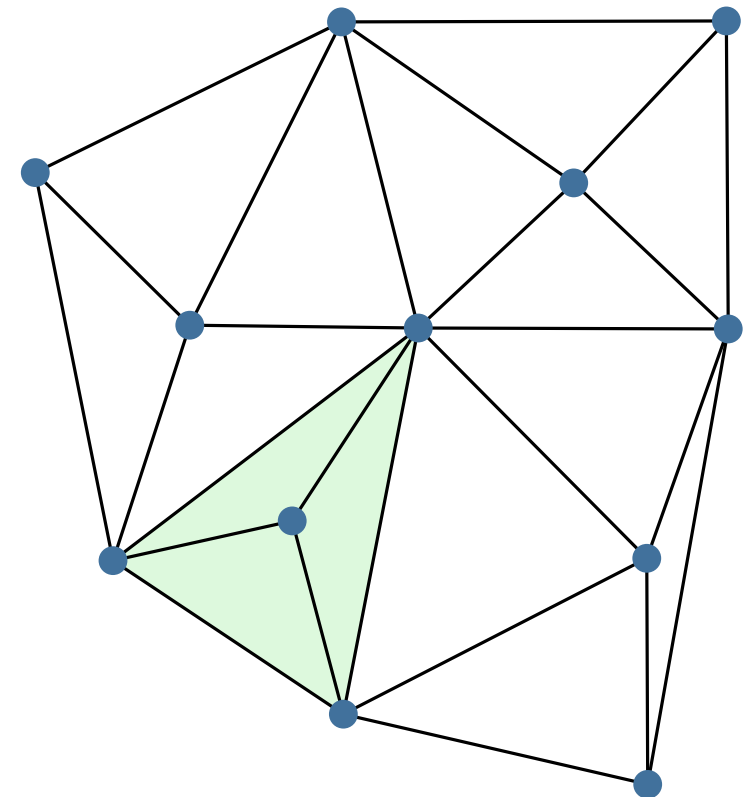
- ▶ number of vertices n
- ▶ weight ratio w : ratio of maximum to minimum vertex weight
- ▶ nesting ratio $nest$: fraction of the points to nest inside triangles



Generation of test data (same as Mchedlidze & Schnorr)

Input:

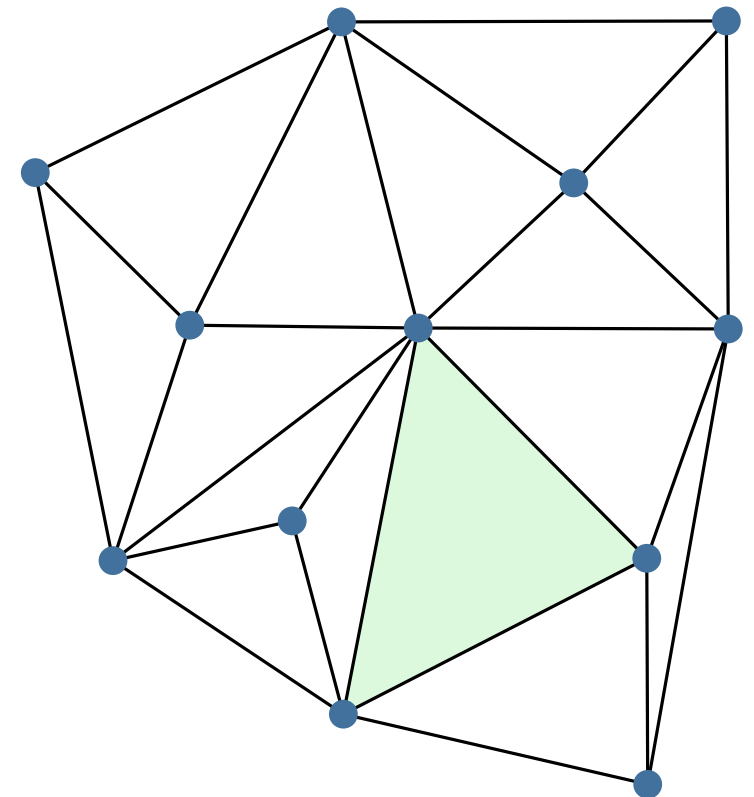
- ▶ number of vertices n
- ▶ weight ratio w : ratio of maximum to minimum vertex weight
- ▶ nesting ratio $nest$: fraction of the points to nest inside triangles



Generation of test data (same as Mchedlidze & Schnorr)

Input:

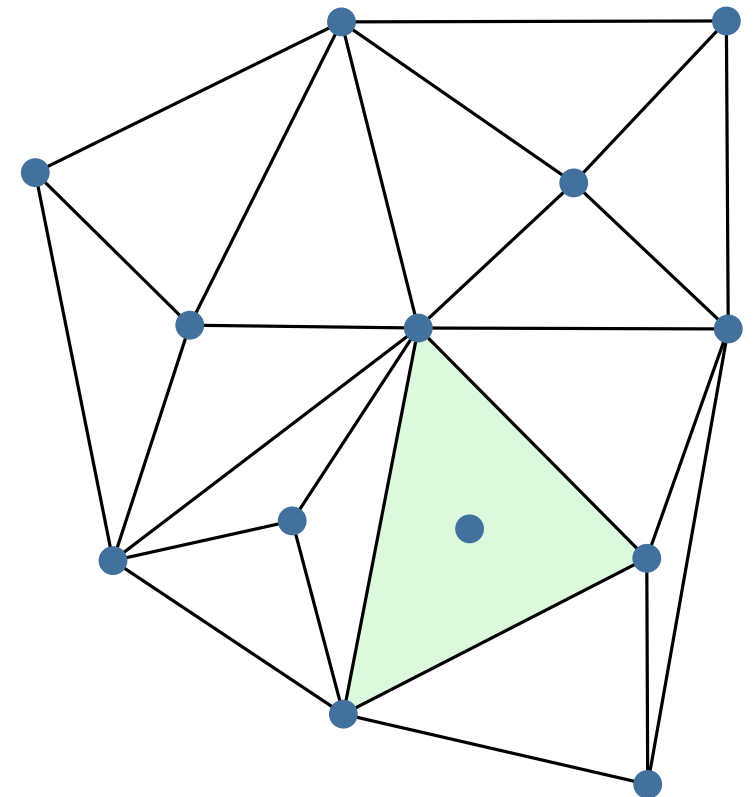
- ▶ number of vertices n
- ▶ weight ratio w : ratio of maximum to minimum vertex weight
- ▶ nesting ratio $nest$: fraction of the points to nest inside triangles



Generation of test data (same as Mchedlidze & Schnorr)

Input:

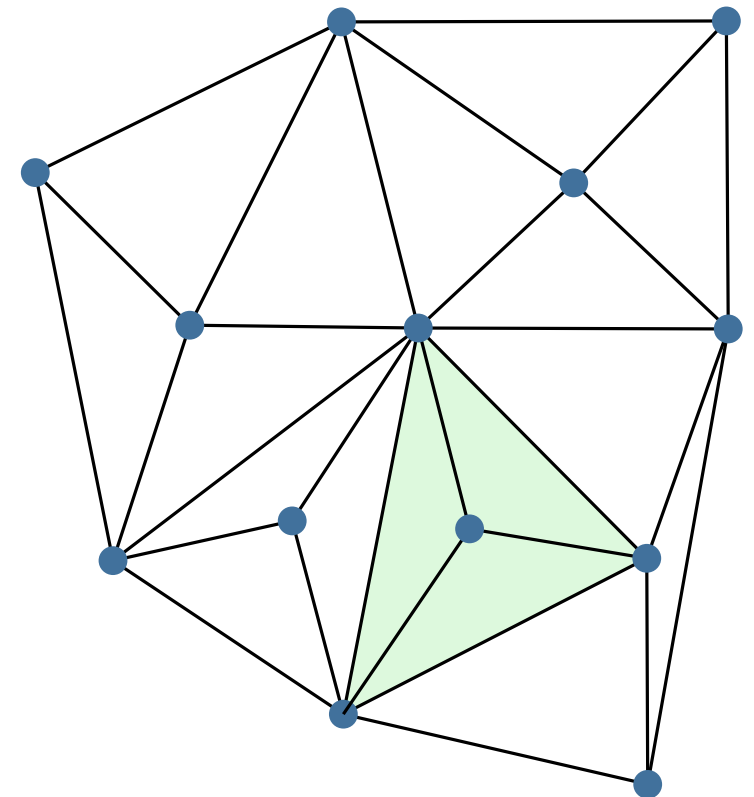
- ▶ number of vertices n
- ▶ weight ratio w : ratio of maximum to minimum vertex weight
- ▶ nesting ratio $nest$: fraction of the points to nest inside triangles



Generation of test data (same as Mchedlidze & Schnorr)

Input:

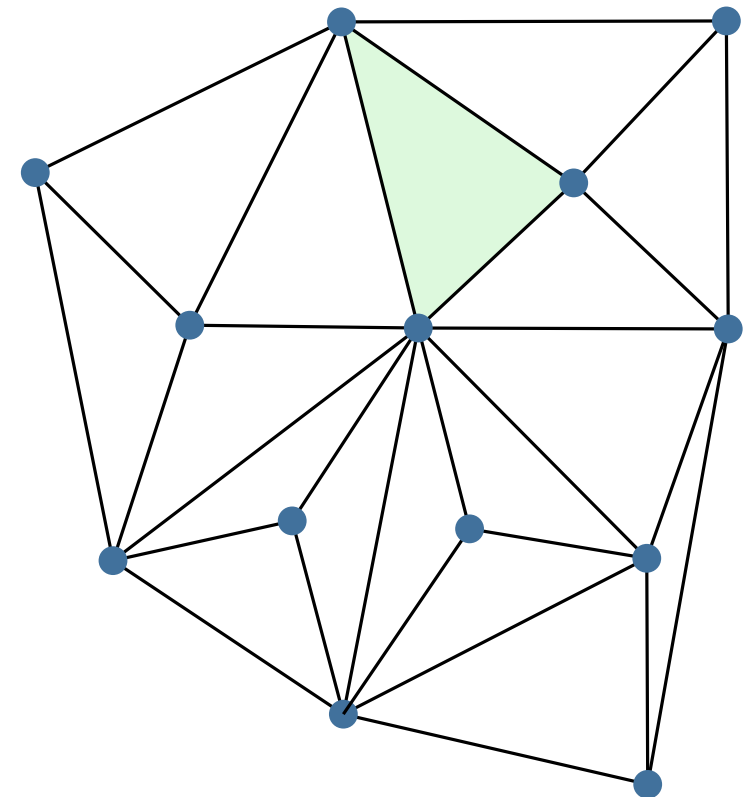
- ▶ number of vertices n
- ▶ weight ratio w : ratio of maximum to minimum vertex weight
- ▶ nesting ratio $nest$: fraction of the points to nest inside triangles



Generation of test data (same as Mchedlidze & Schnorr)

Input:

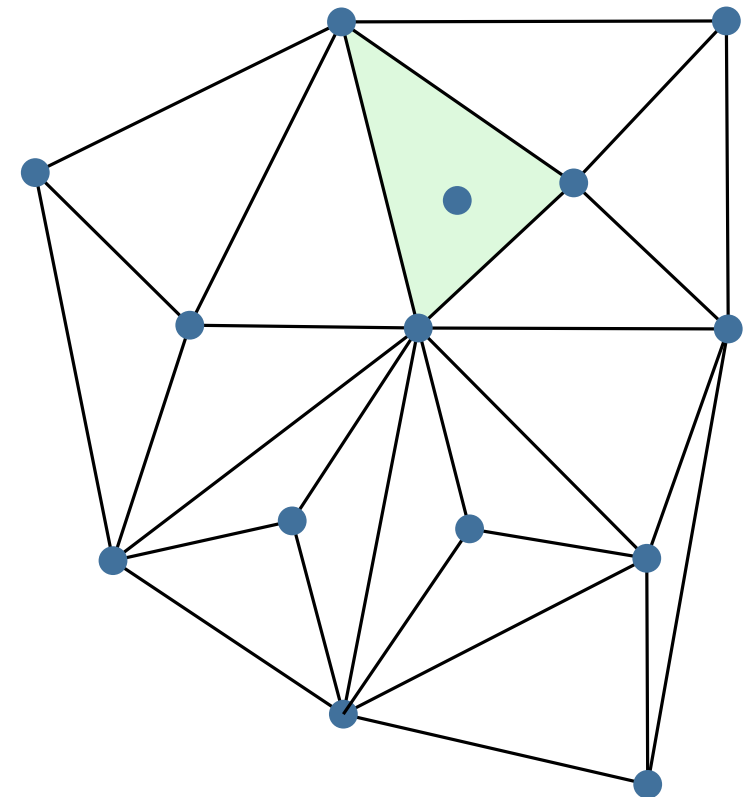
- ▶ number of vertices n
- ▶ weight ratio w : ratio of maximum to minimum vertex weight
- ▶ nesting ratio $nest$: fraction of the points to nest inside triangles



Generation of test data (same as Mchedlidze & Schnorr)

Input:

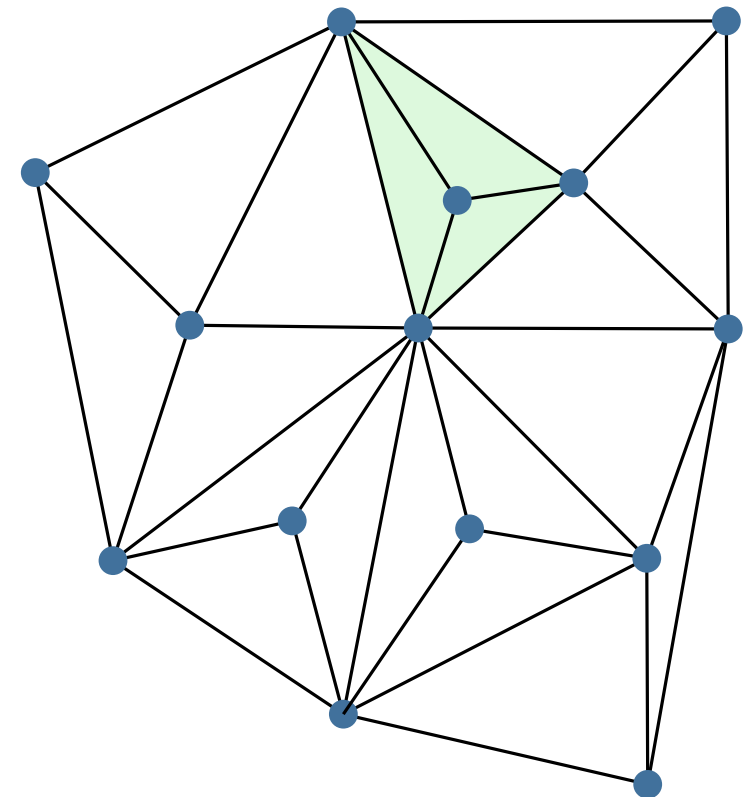
- ▶ number of vertices n
- ▶ weight ratio w : ratio of maximum to minimum vertex weight
- ▶ nesting ratio $nest$: fraction of the points to nest inside triangles



Generation of test data (same as Mchedlidze & Schnorr)

Input:

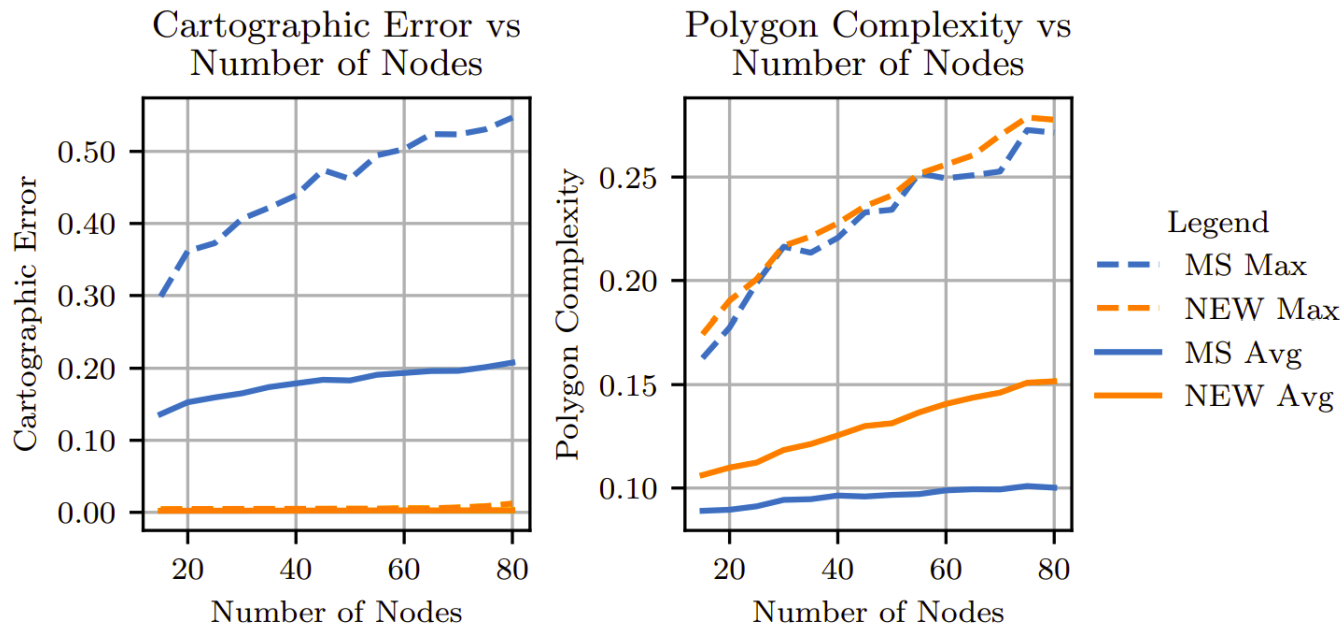
- ▶ number of vertices n
- ▶ weight ratio w : ratio of maximum to minimum vertex weight
- ▶ nesting ratio $nest$: fraction of the points to nest inside triangles



The results

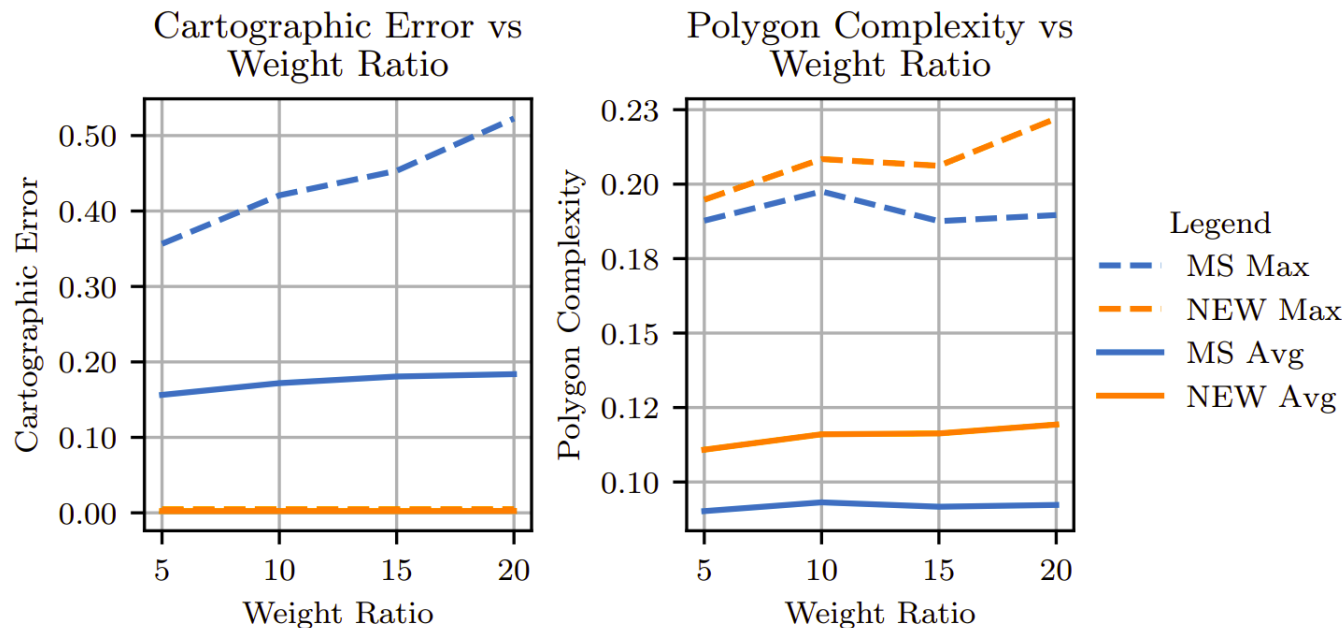
The results

- ▶ number of nodes: $\forall n \in \{15, 20, 25, \dots, 80\}$, we generated 50 graphs with nesting ratio $nest = 0$ and weight ratio $w = 5$



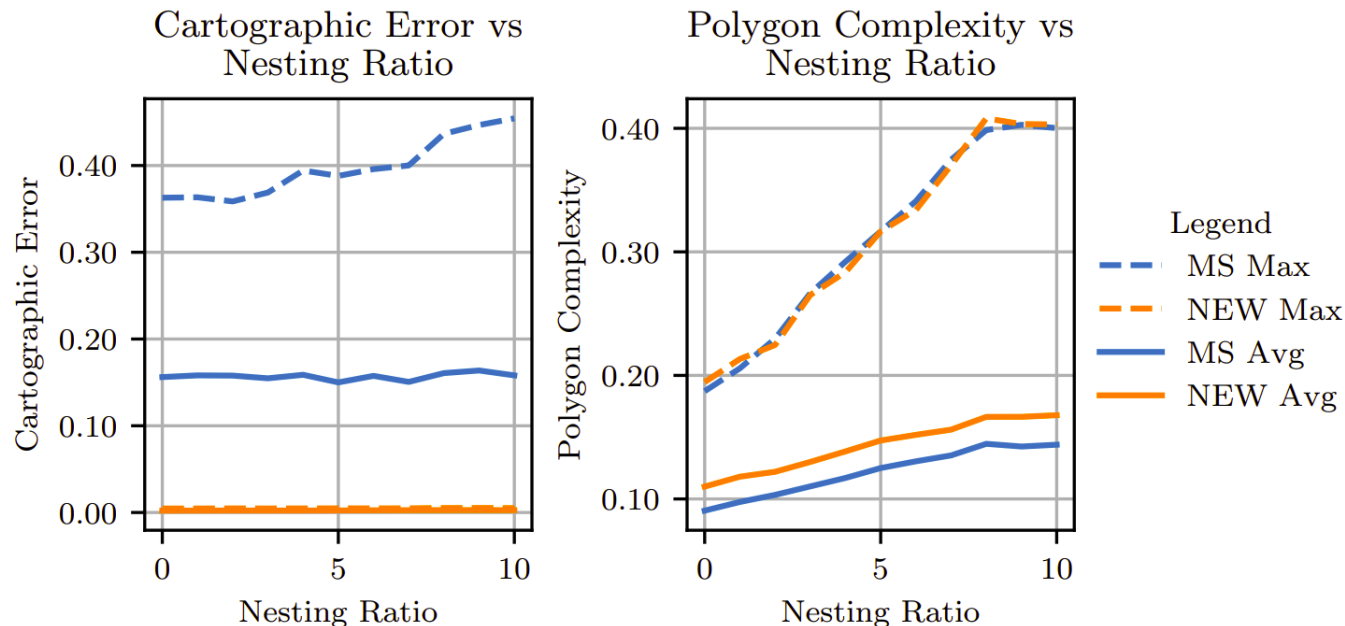
The results

- ▶ number of nodes: $\forall n \in \{15, 20, 25, \dots, 80\}$, we generated 50 graphs with nesting ratio $nest = 0$ and weight ratio $w = 5$
- ▶ weight ratio: $\forall w \in \{5, 10, 15, 20\}$, we generated 50 graphs with $n = 20$ nodes and nesting ratio $nest = 0$



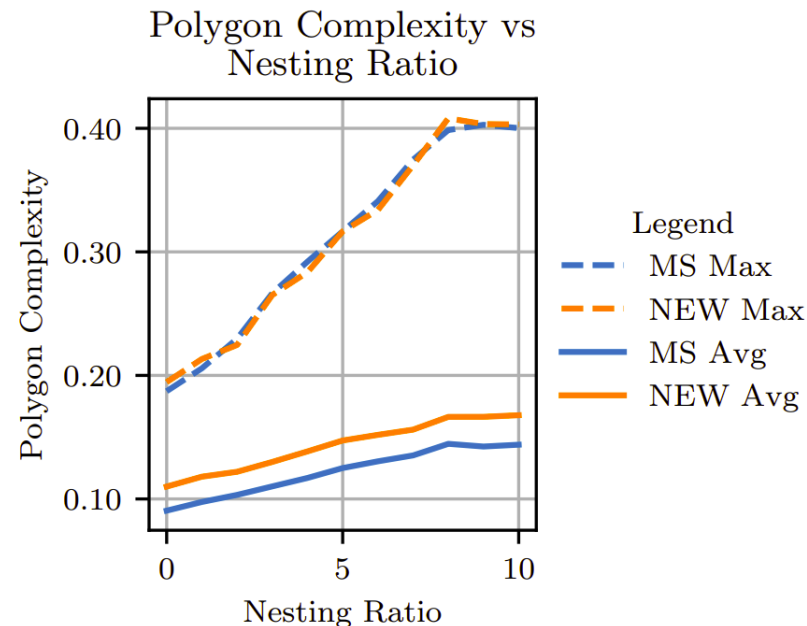
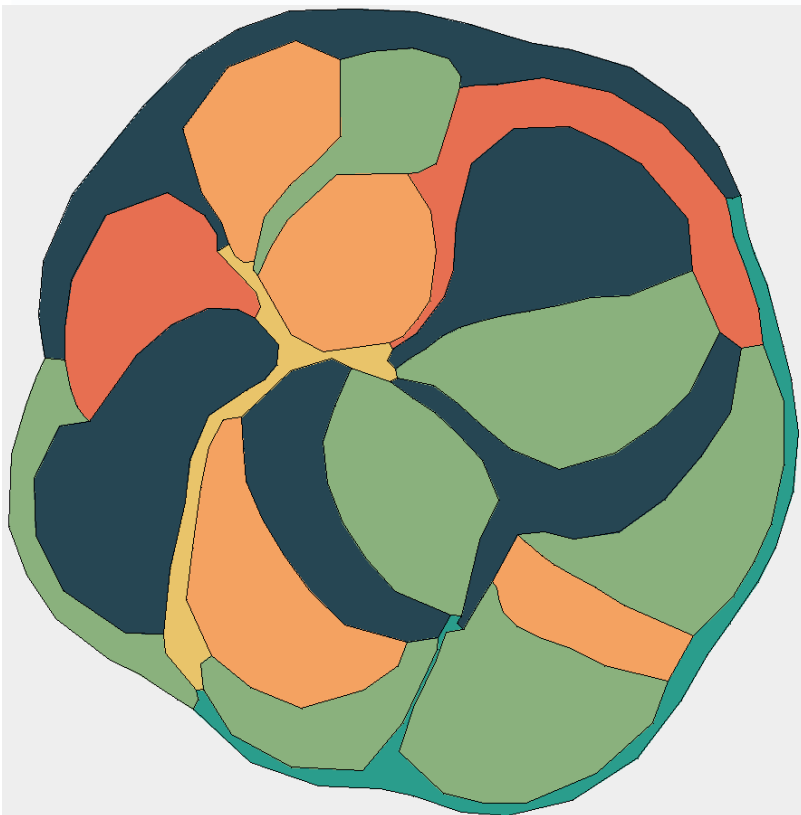
The results

- ▶ number of nodes: $\forall n \in \{15, 20, 25, \dots, 80\}$, we generated 50 graphs with nesting ratio $nest = 0$ and weight ratio $w = 5$
- ▶ weight ratio: $\forall w \in \{5, 10, 15, 20\}$, we generated 50 graphs with $n = 20$ nodes and nesting ratio $nest = 0$
- ▶ nesting ratio: $\forall nest \in \{0, 0.1, 0.2, \dots, 1.0\}$, we generated 50 graphs with $n = 20$ nodes and weight ratio $w = 5$.

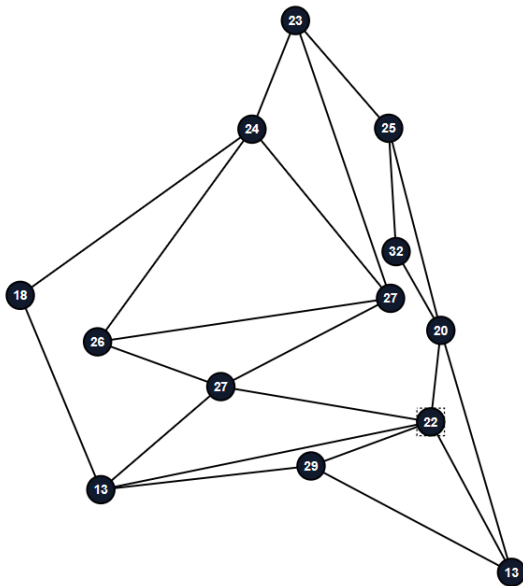


The results

- ▶ number of nodes: $\forall n \in \{15, 20, 25, \dots, 80\}$, we generated 50 graphs with nesting ratio $nest = 0$ and weight ratio $w = 5$
- ▶ weight ratio: $\forall w \in \{5, 10, 15, 20\}$, we generated 50 graphs with $n = 20$ nodes and nesting ratio $nest = 0$
- ▶ nesting ratio: $\forall nest \in \{0, 0.1, 0.2, \dots, 1.0\}$, we generated 50 graphs with $n = 20$ nodes and weight ratio $w = 5$.

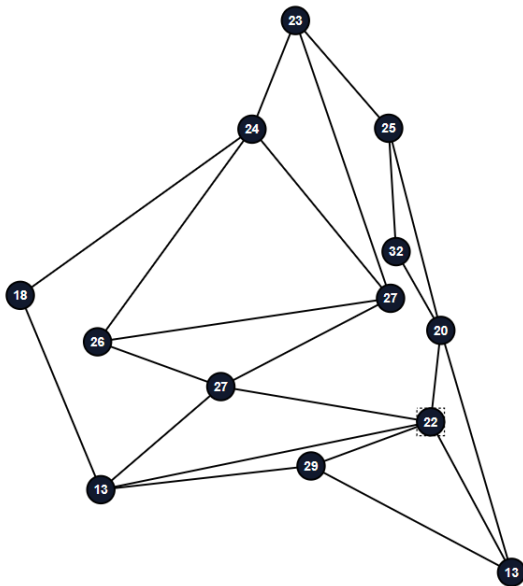


Dealing with non-Triangulated Graphs: Metaphorical maps with holes.



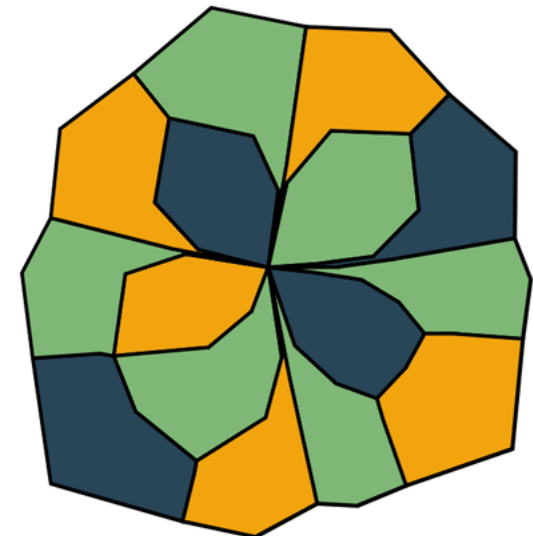
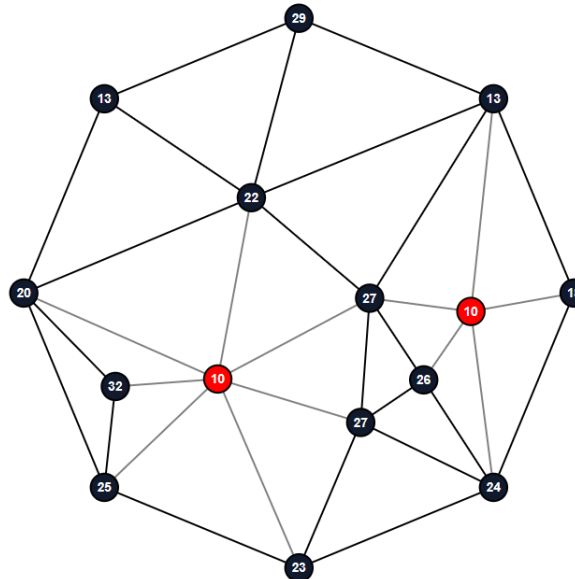
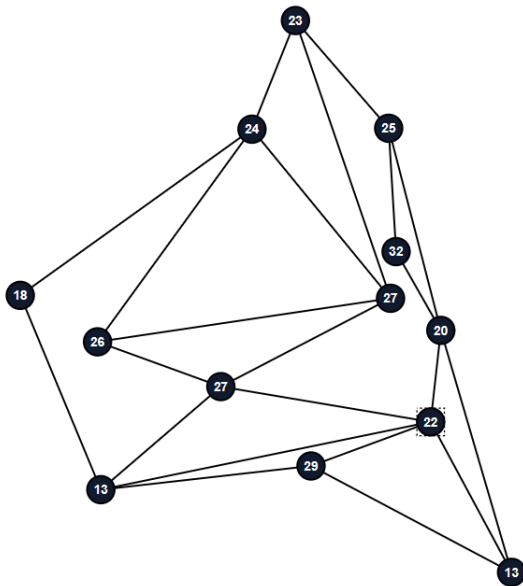
Dealing with non-Triangulated Graphs: Metaphorical maps with holes.

- Step 1: Triangulate graph with auxiliary vertices



Dealing with non-Triangulated Graphs: Metaphorical maps with holes.

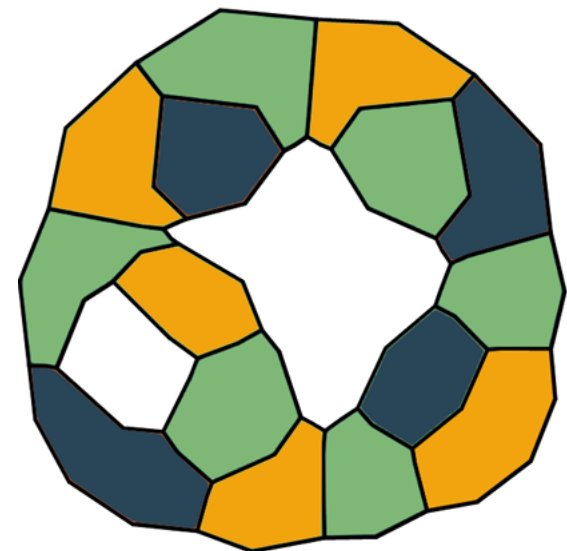
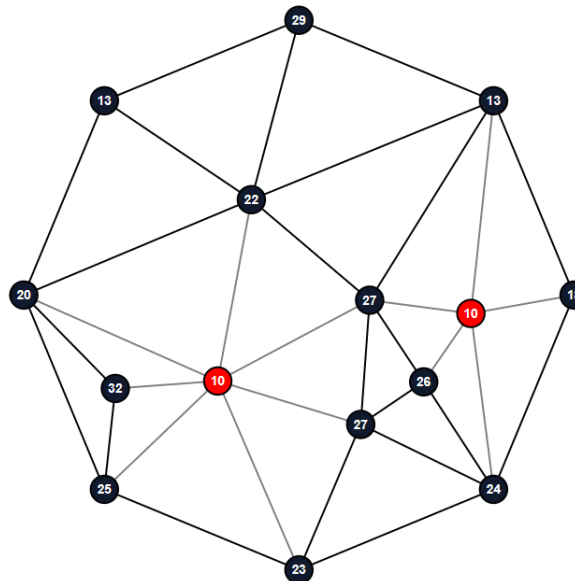
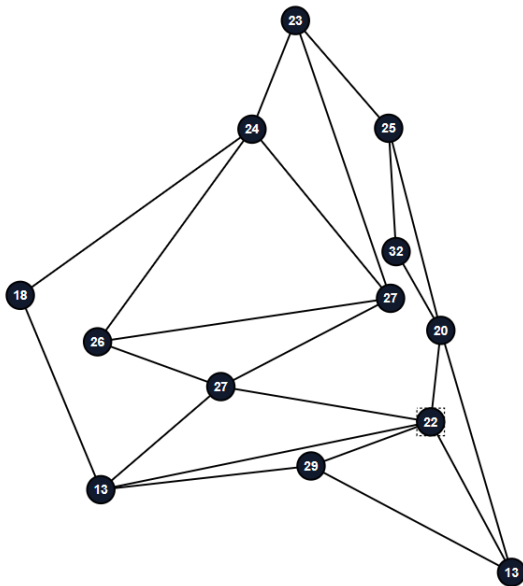
- ▶ Step 1: Triangulate graph with auxiliary vertices
- ▶ Step 2: Apply Tutte's barycentric embedding algorithm



Dealing with non-Triangulated Graphs: Metaphorical maps with holes.

- ▶ Step 1: Triangulate graph with auxiliary vertices
- ▶ Step 2: Apply Tutte's barycentric embedding algorithm
- ▶ Step 3: Assign to every auxiliary vertex the weight

$$\text{holeWeight}(h_i) = \frac{1}{4\deg(h_i)} \left[\sum_{(v_j, h_i) \in E(G)}^j \sqrt{w(v_j)} \right]^2$$



Contribution

- ▶ Improvement of Mchedlidze & Schnorr algorithm
 - Simple-looking metaphorical maps with near perfect cartographic accuracy
 - Metaphorical maps with holes

Contribution

- ▶ Improvement of Mchedlidze & Schnorr algorithm
 - Simple-looking metaphorical maps with near perfect cartographic accuracy
 - Metaphorical maps with holes

Research directions

- ▶ better quality metrics?
- ▶ new weight functions for holes?

Contribution

- ▶ Improvement of Mchedlidze & Schnorr algorithm
 - Simple-looking metaphorical maps with near perfect cartographic accuracy
 - Metaphorical maps with holes

Research directions

- ▶ better quality metrics?
- ▶ new weight functions for holes?



Demo application



Experiment benchmark

Contribution

- ▶ Improvement of Mchedlidze & Schnorr algorithm
 - Simple-looking metaphorical maps with near perfect cartographic accuracy
 - Metaphorical maps with holes

Thank you for your attention!

Research directions

- ▶ better quality metrics
- ▶ new weight functions for holes?

Questions?



Demo application



Experiment benchmark